

SUMMER INTERNSHIP PROJECT REPORT

By

Youssef HASNAOUI

Running Coremark on STM32 devices using Csolution project structure

Professional supervisor:

Mohamed HAMROUNI

Project Elaborated Within STMicroelectronics



Dedications

I dedicate this work to my family and friends.

*To my mother **Monia Sellami**, your love and support means so much to me in every step of my journey. You're the best.*

*To my father **Makram Hasnaoui**, thank you so much for all the sacrifices you've made for my sake. I could never be here without you.*

*To my sister **Aicha**, your kindness and your belief in me have been a constant source of inspiration, and for that, I am very grateful.*

To my friends, you have my most sincere gratitude for your support, your guidance and your company throughout this journey.

Thank you,

Youssef HASNAOUI

Acknowledgements

I would like to sincerely thank everyone who had a hand in the development of this project.

*Firstly, I want to express my utmost appreciation to **Mohamed HAMROUNI**, his guidance and support has been invaluable, and has kept me motivated and determined to give my best effort.*

Last but not least, to every single person that contributed to the successful completion of this project with their constant support and motivation, you have my utmost thanks.

Contents

Introduction	1
1 Internship Context	2
1.1 Host Organization	3
1.1.1 STMicroelectronics	3
1.1.2 Activity Sectors	3
1.1.3 Global Presence	4
1.1.4 ST Tunis	4
1.1.5 ST Support Solutions	5
1.2 Project Context	5
1.3 Problem Statement	6
1.4 State of the Art	6
1.5 Critique of Current State of the Art	7
1.6 Proposed Solution	7
1.7 Objective	8
1.8 Conclusion	8
2 Theory and Key Concepts	9
2.1 STM32 Microcontrollers (MCUs)	10
2.1.1 STM32 Series	10
2.1.2 ARM Cortex-M	10
2.1.3 STM32 Families	10
2.1.4 STM32 UART Peripheral	10
2.1.5 STM32Cube Firmware Package	11
2.2 Benchmarking	11
2.2.1 Overview	11
2.2.2 Benchmarking Microcontrollers	12
2.2.3 Coremark	12
2.3 C/C++ Build Process	14
2.3.1 Compilation Toolchain	15
2.3.2 Embedded Systems Toolchains	15
2.3.3 Build Runners	16

2.3.4	Build Systems	17
2.4	Open-CMSIS-Pack	17
2.4.1	CMSIS-Packs	17
2.4.2	CMSIS-Pack Format	18
2.4.3	Software Components	19
2.4.4	CMSIS Solution Project Structure	20
2.5	Project Generation	21
2.5.1	Regular Expressions:	21
2.5.2	Templates & File Manipulation	22
3	Objectives Specification and Work Environment	24
3.1	Project Specification	25
3.1.1	Functional Requirements	25
3.1.2	Non-Functional Requirements	25
3.2	Structural Decisions	26
3.2.1	CoreMark Project	26
3.2.2	Project Generation Tool	27
3.3	Use-Case Diagram	28
3.4	Hardware Resources	30
3.5	Software Resources	33
3.5.1	Programming Languages	33
3.5.2	Integrated Development Environments and Text Editors	33
3.5.3	Compiler Toolchains	34
3.5.4	Development Tools	35
4	Solution Implementation	38
4.1	Project Development Overview	39
4.1.1	Porting CoreMark On STM32 Devices	39
4.1.2	Running CoreMark Using Csolution Project Structure	42
4.1.3	Project Structure Definition	42
4.1.4	Automation Analysis	42
4.1.5	Fully Automated Generation	44
4.1.6	Project Validation	45
4.2	CoreMark Project Architecture	45
4.2.1	User Code	45

4.2.2	Modifiable Files	46
4.2.3	Static Folders	47
4.2.4	Csolution Files	49
4.3	Project Generation Tool	52
4.3.1	Overview	52
4.3.2	Device Configuration	56
4.3.3	UART Configuration	57
4.3.4	Extra Configuration	57
4.3.5	Firmware Management	57
4.3.6	File Extraction	57
4.3.7	File Processing	58
4.3.8	Template Generation	58
4.3.9	Build Information Generation	58
4.3.10	Error Handling And Resilience	59
4.3.11	Extensibility And Future Proofing	59
	Conclusion	61

List of Figures

1.1	STMicroelectronics Activity Sectors	3
1.2	STMicroelectronics Worldwide Presence	4
1.3	STMicroelectronics Tunis	5
2.1	Distribution of control instructions and mispredictions over CoreMark execution.	14
2.2	Compilation Process Diagram	15
2.3	Software Packs Types	18
2.4	Software Component Interface	19
2.5	CMSIS Solution Project Structure	20
2.6	Templating Process	22
3.1	Project Generation Tool Architecture	27
3.2	Automated CoreMark Benchmarking Use-Case Diagram	28
3.3	STM32 Microcontroller Portfolio	30
3.4	Csolution Operation	36
3.5	Cbuild Workflow	37
4.1	CoreMark Timing Start Function	40
4.2	CoreMark Timing End Function	40
4.3	CoreMark ee_printf Function Definition	41
4.4	Function to send a single character over UART	41
4.5	Sample CoreMark Results Transmitted over UART	41
4.6	Overview of the CoreMark project structure	42
4.7	cm_gen Sequence Diagram	44
4.8	CoreMark Project Architecture Overview	45
4.9	csolution file contents	49
4.10	Extract of cproject file contents	50
4.11	Extract of cdefault file contents	51
4.12	cm_gen DataFlow Diagram	52
4.13	Sequence Diagram describing cm_gen's internal architecture	53

List of Tables

1.1	Comparison of Approaches for Embedded Benchmarking and Automation	7
3.1	Functional Requirements for CMSIS-Toolbox based automated benchmarking	25
3.2	CoreMark Project Structure	27
3.3	Project Generation Tool Structure	28
3.4	Automated CoreMark Benchmarking	28
3.5	STM32 Microcontroller Families	31
3.6	Relevant VSCode Extensions	34
4.1	Automatable Patterns in CoreMark Project Generation	43
4.2	HAL Modules Used in the CoreMark Project	47
4.3	CoreMark Source Files	48
4.4	Project Generation Tool options	54

List of Acronyms

- **ABI** = Application **B**inary **I**nterface
- **API** = Application **P**rogramming **I**nterface
- **ARM** = Advanced **RISC** **M**achine
- **BRE** = **B**asic **R**egular **E**xpression
- **BSP** = **B**oard **S**upport **P**ack
- **BSS** = **B**lock **S**tarted by **S**ymbol
- **CFSR** = **H**ard**F**ault **S**tatus **R**egister
- **CLI** = **C**ommand-**L**ine **I**nterface
- **CMSIS** = **C**ortex **M**icrocontroller **S**oftware **I**nterface **S**tandard
- **CPU** = **C**entral **P**rocessing **U**nit
- **DAP** = **D**ebug **A**dapter **P**ort
- **DFP** = **D**evice **F**amily **P**ack
- **DRAM** = **D**ynamic **R**andom **A**ccess **M**emory
- **DSP** = **D**igital **S**ignal **P**rocessor
- **DWARF** = **D**ebugging **W**ith **A**ttributed **R**ecord **F**ormats
- **EEMBC** = **E**mbedded **M**icroprocessor **B**enchmark **C**onsortium
- **ELF** = **E**xecutable and **L**inkable **F**ormat
- **ERE** = **E**xtended **R**egular **E**xpression
- **FPU** = **F**loating **P**oint **U**nit
- **FPU** = **F**loating-**P**oint **U**nit
- **GCC** = **G**NU **C**ompiler **C**ollection
- **GDB** = **G**NU **D**ebugger
- **GNU** = **G**NU's **N**ot **U**nix

- **GUI** = **G**raphical **U**ser **I**nterface
- **HAL** = **H**ardware **A**bstraction **L**ayer
- **HFSR** = **H**ard**F**ault **S**tatus **R**egister
- **I/O** = **I**nterface **O**utput
- **IAR** = **I**ngenjör**s**firm**a** **A**nders **R**undgren
- **IDE** = **I**ntegrated **D**evelopement **E**nvironment
- **IEEE** = **I**nstitute of **E**lectrical and **E**lectronics **E**ngineers
- **IP** = **I**ntegrated **P**eripheral
- **IV** = **I**nitialization **V**ector
- **JSON** = **J**ava**S**cript **O**bject **N**otation
- **JTAG** = **J**oint **T**est **A**ction **G**roup
- **LL** = **L**ow-**L**ayer
- **LLVM** = **L**ow **L**evel **V**irtual **M**achine
- **MCU** = **M**icro **C**ontroller **U**nit
- **MFLOPS** = **M**ega **F**loating-point **O**perations **P**er **S**econd
- **MOPS** = **M**ega **O**perations **P**er **S**econd
- **MVE** = **M**-Profile **V**ector **E**xtension
- **OCD** = **O**n-**C**hip **D**ebugger
- **PLM** = **P**roject **L**ifetime **M**anagement
- **POSIX** = **P**ortable **O**perating **S**ystem **I**nterface
- **RAM** = **R**andom **A**ccess **M**emory
- **ROM** = **R**ead-**O**nly **M**emory
- **RTE** = **R**un**T**ime **E**nvironment
- **RTOS** = **R**ead-**T**ime **O**perating **S**ystem

- **RegEx** = **Regular Expression**
- **SCB** = **System Control Block**
- **SCVD** = **Software Component Viewer Description**
- **SDRAM** = **Synchronous Dynamic Random Access Memory**
- **SRAM** = **Static Random Access Memory**
- **SRE** = **Simple Regular Expression**
- **SVD** = **System View Description**
- **SWD** = **Single-Wire Debug**
- **SWO** = **Serial Wire Output**
- **UART** = **Universal Asynchronous Receiver Transmitter**
- **USART** = **Universal Synchronous Asynchronous Receiver Transmitter**
- **VCP** = **Virtual COM Port**
- **YAML** = **Yet Another Markup Language**

Introduction

In today's technological landscape, the importance of benchmarking in embedded systems cannot be overstated. Embedded systems are integral to a wide range of applications, from consumer electronics and industrial automation to healthcare devices and automotive systems. Therefore, evaluations provide objective data on performance, efficiency and reliability, magnifying their importance when it comes to making informed design and purchasing decisions. As systems grow more complex and interconnected, the number of configurable parameters and potential performance bottlenecks expands, increasing the risk of suboptimal capabilities, wasted resources and failed deployments. Establishing a rigorous and standardized benchmarking methodology is therefore paramount to ensure systems meet their intended requirements and deliver value in both development and production environments.

Benchmarking provides the fundamental framework for this objective analysis by establishing a standardized set of metrics and tests to quantify a system's performance. Effective benchmarking allows developers to measure key characteristics such as processing speed, memory throughput, power consumption, and real-time task execution, creating a data-driven basis for comparison. However, executing benchmarks in embedded systems comes with challenges. Developers must navigate complexities such as isolating the unit under test, minimizing the influence of external systems, selecting appropriate tools, and ensuring that the test conditions are consistent and reproducible across different platforms and devices. Inaccurate methodology or poorly designed tests can cause misleading results, rendering the data useless for making decisions.

The dependency on proprietary and platform-specific vendor tools can lock projects into a single ecosystem and hinder reproducibility. There is a growing need to have development environments that prioritize portability and vendor independence, allowing benchmarks to be built, deployed, and executed consistently across a wide range of hardware. By abstracting away toolchain-specific complexities, such environments allow developers to focus on the benchmark results themselves rather than the intricacies of the different build processes.

This report explores the implementation of the CoreMark benchmark on STM32 devices using CMSIS-Toolbox, and the related Csolution project structure as a step towards this goal. This method demonstrates a move away from proprietary IDE-bound setups and towards a portable, platform-independent driven workflow. By leveraging a toolchain-agnostic approach, we establish a reproducible environment for evaluations. This reduces vendor lock-in, enhances cross-platform compatibility, ensures the data is a reliable reflection of the hardware capabilities, and, as a positive side effect, allows for a fair comparison between the available toolchains.

INTERNSHIP CONTEXT

Contents

1	Host Organization	3
2	Project Context	5
3	Problem Statement	6
4	State of the Art	6
5	Critique of Current State of the Art	7
6	Proposed Solution	7
7	Objective	8
8	Conclusion	8

Internship Context

This opening chapter focuses on introducing the host company STMicroelectronics, and provides an overview of the company's activities and global presence. It also includes an introduction to ST Tunis and the Support Solutions team, which played a pivotal role in the execution of this project, as well as an outline of the project's overall framework. Following the presentation of the challenges, we will explore the current state of the art and finally present the proposed solution which is the main objective of this project.

1.1 Host Organization

1.1.1 STMicroelectronics

STMicroelectronics (ST) is a global leader in the semiconductor industry, providing innovative solutions across various markets, including automotive, industrial, personal electronics, and communications equipment.

Founded in 1987 through the merger of SGS Microelettronica of Italy and Thomson Semiconducteurs of France, ST has grown to become one of the largest semiconductor companies in the world.

1.1.2 Activity Sectors

ST's mission is to be the undisputed leader in the semiconductor market, delivering sustainable and innovative solutions that make a positive impact on people's lives. The company's vision is to create technology that enables a more intelligent and connected world, driving progress in key areas such as smart driving, smart industry, smart home, and smart city applications.

The following illustration highlights the key end markets targetted by ST's solutions.



Figure 1.1: STMicroelectronics Activity Sectors

Figure 1.2 illustrates the company's global network, which includes at least 100





Figure 1.3: STMicroelectronics Tunis

1.1.5 ST Support Solutions

The ST Support Solutions team is dedicated to delivering comprehensive customer support while managing and publishing STM32 product documentation. Their role includes providing in-depth technical support for ST's tools and products, helping customers troubleshoot problems and optimize their use of these technologies. They ensure the accuracy, clarity, and consistency of all technical documentation, which is regularly updated to reflect the latest developments and innovations. The team is also involved in enhancing and maintaining existing products, as well as defining new products, derivatives, and application references to meet evolving market needs. Additionally, they lead initiatives related to intellectual property applications, offering specific support through the development of application notes, training programs, and other resources that help customers fully understand and utilize ST's products. This multifaceted approach ensures both customer satisfaction and continuous improvement in the STM32 ecosystem.

1.2 Project Context

Benchmarking microcontrollers is a fundamental step in evaluating their computational capabilities, power efficiency, and suitability for various applications, such as IoT, robotics, or industrial control. Among the many available benchmarks, CoreMark, developed by EEMBC, has become the de facto standard for embedded systems because it provides a well-defined, portable, and reliable performance metric. STM32 microcontrollers, are widely adopted in the embedded systems industry due to their scalability, rich peripheral set, and performance-to-cost ratio.

The **CMSIS** initiative, led by Arm, and its Open-CMSIS-Pack ecosystem aim to standardize microcontroller development workflows. Within this ecosystem, Csolution provides a modern, metadata-driven project structure

that enables cross-platform builds, device abstraction, and automation. Leveraging Csolution can significantly simplify benchmarking workflows by providing a unified and reproducible project generation and build process.

1.3 Problem Statement

Running CoreMark on STM32 devices traditionally requires manual setup for each device, including:

- Creating and configuring individual projects for different STM32 families.
- Managing peripheral initialization.
- Providing a clock source to Coremark.
- Handling different compiler options and toolchains.
- Customizing linker scripts manually.
- Maintaining multiple project files for IDEs like Keil μ Vision, IAR EWARM, or others.

This manual process is time-consuming, error-prone, and non-scalable, especially when benchmarking a wide range of devices. There is currently no standardized, automated workflow that allows developers to quickly generate CoreMark-ready projects for different STM32 targets while ensuring consistency and portability.

1.4 State of the Art

The current approaches to running benchmarks on STM32 devices typically rely on:

- **Vendor-Specific IDEs:** IDEs provide an easy way to manage projects via a graphical interface, they typically provide tools for peripheral configuration, memory management and compiler options. They also offer integrated build and debug environments.
- **Standalone CoreMark Implementations:** EEMBC provides CoreMark source code with minimal reference implementations, it is up to the developer to manually adapt it to the target device, providing startup and initialization code, memory configuration and a clock source.
- **Custom Build Systems:** Some environments require the use of custom build systems such as CMake and Make, these tools allow the developer to manually manage dependencies, defines and other C/C++ related build configuration.
- **CMSIS Packs:** CMSIS packs offer a standardized way of packaging drivers, middleware and device specific files, they provide metadata to describe the device's memory and peripheral layout.

1.5 Critique of Current State of the Art

While the above methods work, they exhibit several limitations:

Table 1.1: Comparison of Approaches for Embedded Benchmarking and Automation

Approach	Advantages	Limitations
Vendor-Specific IDEs	Intuitive GUI, built-in drivers, easy to use	Projects are IDE-specific, hard to automate, poor scalability
Standalone CoreMark	Portable reference code on computers	Significant manual effort for adaptation to each MCU
Custom Build Systems	Flexible, automation-friendly	Requires deep expertise, no standardized metadata integration
CMSIS Packs	Standardized packaging and metadata support	Lack seamless integration with benchmarking and automation

1.6 Proposed Solution

The proposed solution centers around the aspects of automation and standardization, leveraging the **CMSIS Solution project structure**, part of the **Open-CMSIS-Pack** ecosystem in order to achieve the desired outcome.

The key aspects of the solution are:

- **Unified Project structure:** By making use of the `csolution` project files, we can describe build configuration, dependencies, and device parameters in a centralized way, simplifying maintenance and reuse.
- **Automating Project Generation:** While the **Csolution project structure** offers a standardized manner of configuring projects, the source code of the application has to be managed in some way, although generators like **STM32CubeMX** does a decent job of generating peripheral configuration, it is still slow and GUI based, so it can't be used in a fully automated environment. Therefore, we can make use of creating a tool to handle automating source code and linker script generation, device configuration and even generating the `csolution` project files.
- **Cross-Toolchain Support:** Allow the project to be compiled by the main toolchains used in embedded applications, through CMSIS-Toolbox, as well as ensuring the project can be built and debugged in multiple IDE environments.
- **CoreMark Integration:** Embed the CoreMark source code into the standardized generation template, making it easily portable to all STM32 MCU targets.

1.7 Objective

The main objective of this project is to develop a standardized, automated workflow for running CoreMark on STM32 devices using the Csolution project structure.

1.8 Conclusion

Benchmarking embedded systems is essential for performance evaluation and hardware selection. However, the current manual approaches to running CoreMark on STM32 devices are inefficient, inconsistent, and hard to scale.

This project addresses these challenges by introducing a Csolution-based workflow that integrates device metadata, automation, and portable build configurations. By leveraging the Open-CMSIS-Pack ecosystem, the proposed solution provides a unified, reproducible, and scalable benchmarking framework.

The outcome is a streamlined process where CoreMark can be quickly deployed to any STM32 target, results can be gathered reliably, and the entire workflow can serve as a foundation for future benchmarking or performance evaluation tasks across diverse embedded platforms.

THEORY AND KEY CONCEPTS

Contents

1	STM32 Microcontrollers (MCUs)	10
2	Benchmarking	11
3	C/C++ Build Process	14
4	Open-CMSIS-Pack	17
5	Project Generation	21

Introduction

In this chapter, we will explore the key concepts essential to our project. We begin by introducing the STM32 ecosystem, providing an overview of its components and their functionalities. Following this, we delve into the theoretical aspects of benchmarking and embedded build systems relevant to our work. We then present the open-CMSIS-pack ecosystem. Finally we will discuss the essentials of automating project generation.

2.1 STM32 Microcontrollers (MCUs)

2.1.1 STM32 Series

STM32 MCUs are a family of 32-bit microcontrollers based on the ARM Cortex-M processor. Developed by STMicroelectronics, the STM32 series offers a wide range of products that cater to various applications, from simple embedded systems to complex industrial automation. The STM32 MCUs are categorized into several series, each designed to meet specific application requirements.

2.1.2 ARM Cortex-M

The STM32 microcontrollers are built around the ARM Cortex-M cores, a group of 32-bit RISC ARM processors cores optimized for low-cost and power-efficient integrated circuits. The ARM Cortex-M family includes several cores, each offering different levels of performance and features.

2.1.3 STM32 Families

STM32 MCUs are grouped into families, a combination of a series and an ARM Cortex-M core, the number proceeding the STM32 series indicates the core, for example, STM32H7 family indicates a microcontroller from the STM32H series with a Cortex-M7 core, while the STM32N6 family indicates a STM32N series microcontroller with a Cortex-M55 core. Refer to Table 3.5 for a full list of STM32 MCU families.

2.1.4 STM32 UART Peripheral

STM32 microcontrollers come with a rich set of integrated peripherals. The one of interest in this report is the UART peripheral. UART is a hardware protocol and communication interface that uses two wires for data exchanges between peripherals that do not share a common clock signal. It fits our use case of transmitting data from the microcontroller to the host computer in order to have access to Coremark results.

2.1.5 STM32Cube Firmware Package

The STM32Cube Firmware Package is a comprehensive software suite provided by STMicroelectronics to accelerate development on STM32 microcontrollers. Its components include but are not limited to:

- **CMSIS:** A vendor-independent standard defined by Arm that provides a consistent API for Cortex-M cores, STM32Cube Firmware provides an implementation of the CMSIS-Core layer, these include device and family specific header files, as well as templates for startup files and linker scripts.
- **HAL:** Simplifies peripheral configuration and access through high-level APIs, reducing development complexity and allows for re-usability across different hardware.
- **LL Drivers:** Provide fine-grained control of peripherals with minimal overhead.

2.2 Benchmarking

2.2.1 Overview

Benchmarking is the systematic process of evaluating the performance of a computing system, or a specific component within that system, by running a standardized set of tasks and synthetic workloads. The key aspects of benchmarks include:

- **Metrics:** Performance is measured using quantifiable units, Common metrics include:
 - **Throughput:** The amount of work done per unit of time (e.g., operations/second, frames/second, MB/s).
 - **Latency:** The time taken to complete a single operation (e.g., microseconds per operation).
 - **Power Efficiency:** Performance achieved per watt of power consumed (e.g., points per watt, inferences per joule).
 - **Memory Usage:** The amount of memory consumed during a task.
- **Benchmark Types:**
 - **Synthetic Benchmarks:** These are specialized programs designed to stress specific subsystems like the CPU, memory, or GPU. They provide standardized but often abstract results.
 - **Application Benchmarks:** Use real-world software and workloads (e.g., rendering a video file, compiling a large code-base, running a specific game at a set quality). These measure performance in practical, user-facing scenarios.
 - **Micro-benchmarks:** Isolate and test a very specific, low-level operation (e.g., floating-point multiplication speed, memory access latency).

2.2.2 Benchmarking Microcontrollers

Although the same benchmarks can be run on most pieces of technology, the implementation differs slightly for microcontrollers. Computers can delegate tasks such as scheduling and printing to the underlying operating system, which is not the case when it comes to embedded devices. We have to take into account handling threads, I/O operations, memory regions, and code sections. The following steps are necessary to ensure proper benchmarking on a microcontroller:

- **Clock Source Provision:** Benchmarks rely on CPU ticks in order to get an accurate estimate of the time taken to run the workloads, providing access to the tick counter, whether it be the internal System Clock or an external timer, as well as the frequency of said clock is crucial for proper measurements.
- **Print Logic Implementation:** Computers can offer multiple ways of getting data from a program, such as logging them into files or printing them to the standard output, this option is not available on microcontrollers, therefore, we have to provide our own mechanism of data transmission.
- **Code Execution Management:** For high-end systems relying on an operating system, the most common approach is to load the program from disk into volatile memory (RAM) and begin execution without any extra steps. However, in embedded systems, we have to specify the regions of memory where each part of an application should go, and, due to multiple difference, such as wait-states, clock synchronization and the communication interface, the results can have vastly varying results depending on the chosen regions.

2.2.3 Coremark

There have been many attempts to provide a single number that can totally quantify the ability of a CPU. Be it MHz, MOPS, MFLOPS - all are simple to derive but misleading when looking at actual performance potential. EEMBC's CoreMark is a benchmark that measures the performance of microcontrollers (MCUs) and central processing units (CPUs) used in embedded systems. It is designed to run on devices from 8-bit microcontrollers to 64-bit microprocessors. CoreMark ties a performance indicator to execution of simple code, but rather than being entirely arbitrary and synthetic, the code for the benchmark uses basic data structures and algorithms that are common in practically any application

Coremark Composition

To appreciate the value of CoreMark, it's worthwhile to dissect its composition, which in general is comprised of lists, strings, and arrays (matrixes to be exact). Lists commonly exercise pointers and are also characterized by non-serial memory access patterns. In terms of testing the core of a CPU, list processing predominantly tests how fast data can be used to scan through the list. For lists larger than the CPU's available cache, list processing

can also test the efficiency of cache and memory hierarchy.

2.2.3.1 List Processing

List processing consists of reversing, searching or sorting the list according to different parameters, based on the contents of the list data items. In particular, each list item can either contain a pre-computed value or a directive to invoke a specific algorithm with specific data to provide a value during sorting. To verify correct operation, CoreMark performs a 16b cyclic redundancy check (CRC) based on the data contained in elements of the list. Since CRC is also a commonly used function in embedded applications, this calculation is included in the timed portion of the CoreMark.

2.2.3.2 Matrix Processing

Many algorithms use matrixes and arrays, warranting significant research on optimizing this type of processing. These algorithms test the efficiency of tight loop operations as well as the ability of the CPU and associated toolchain to use ISA accelerators such as MAC units and SIMD instructions. These algorithms are composed of tight loops that iterate over the whole matrix. CoreMark performs simple operations on the input matrixes, including multiplication with a constant, a vector, or another matrix. CoreMark also tests operating on part of the data in the matrix in the form of extracting bits from each matrix item for operations. To validate that all operations have been performed, CoreMark again computes a CRC on the results from the matrix test.

2.2.3.3 State machine processing

An important function of a CPU core is the ability to handle control statements other than loops. A state machine based on switch or ‘if’ statements is an ideal candidate for testing that capability. There are 2 common methods for state machines – using switch statements or using a state transition table. Because CoreMark already utilizes the latter method in the list processing algorithm to test load/store behavior, CoreMark uses the former method, switch and ‘if’ statements, to exercise the CPU control structure. The state machine tests an input string to detect if the input is a number, if it is not a number it will reach the “invalid” state. This is a simple state machine with 9 states. The input is a stream of bytes, initialized to ensure we pass all available states, based on an input that is not available at compile time. The entire input buffer is scanned with this state machine.

2.2.3.4 CoreMark Profiling

Since CoreMark contains multiple algorithms, it is interesting to demonstrate how the behavior changes over time. For example, looking at the percentage of control code executed (samples taken at each 1000 cycles) and branch mis-predictions in Figure 2.1, it is obvious where the matrix algorithm is being called. This is portrayed by the low mis- prediction rate and high percentage of control operations, indicative of tight loops) (for example,

between points 330-390).

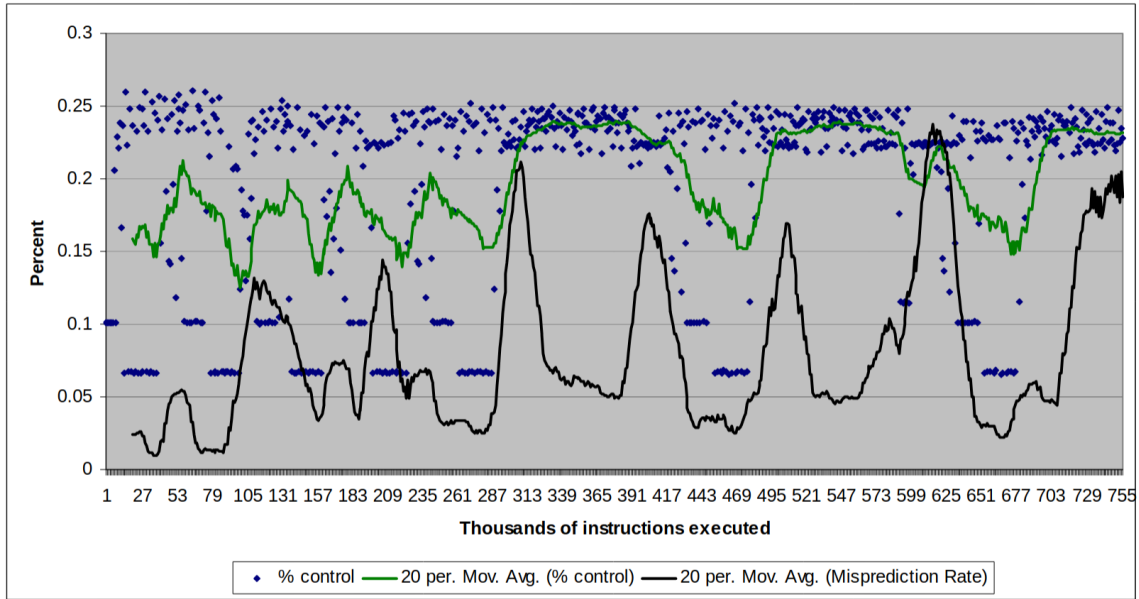


Figure 2.1: Distribution of control instructions and mispredictions over CoreMark execution.

Overall CoreMark is well suited to comparing embedded processors. It is small, highly portable, well understood, and highly controlled. CoreMark verifies that all computations were completed correctly during execution, which helps debug any issues that may come up. The run rules are clearly defined and reporting rules are enforced on the CoreMark web site.

2.3 C/C++ Build Process

The construction of executable firmware for embedded systems necessitates interventions throughout the compilation toolchain, this requirement is driven by the absence of standardized hardware and resource constraints. Consequently, in our context, the build process is characterized by explicit configuration at each stage, including compiler optimizations, targeted assembly integration, and memory management via linker scripts.

2.3.1 Compilation Toolchain

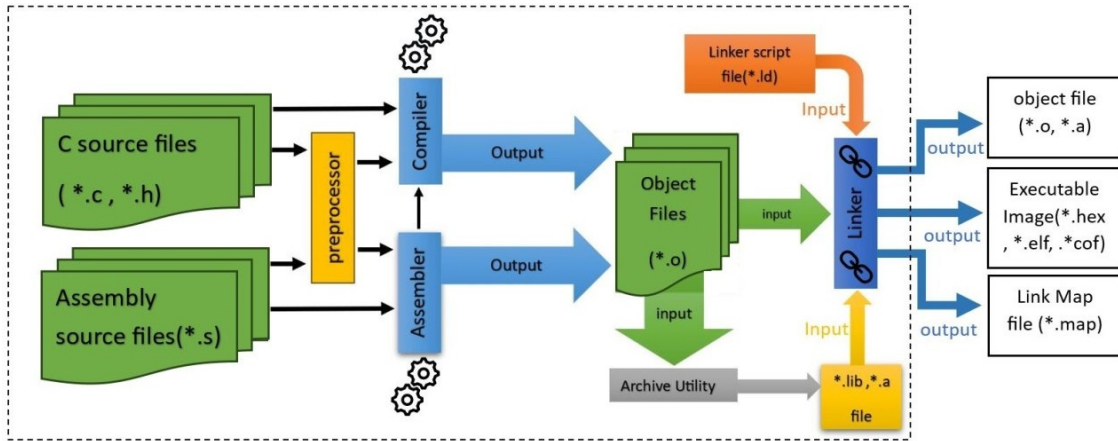


Figure 2.2: Compilation Process Diagram

Going from source code in C/C++ to executable firmware involves multiple steps, apart from the linking stage, all other processes in the toolchain target a single file at a time.

- **Preprocessor** : Preprocessing is the first step of the build process, it expands macros and resolves defines as well as stripping comments from the code.
- **Assembler** : The assembler translates the high-level code into assembly, which is the human-readable format of machine instruction.
- **Compiler** : Although the we use the term compilation as an equivalent to the whole process, compilation at its core is translating code into machine instructions, this step is what translates assembly mnemonics into machine code in a format known as object files.
- **Linker** : The linking stage is the final and most important part of the process, it takes the object files and combines them into a single runnable application.

2.3.2 Embedded Systems Toolchains

While a native toolchain produces binaries that run directly on the host machine, a toolchain dedicated to embedded systems must target a separate device with its own architecture, memory constraints, and hardware interfaces. This difference introduces several additional requirements beyond those of a regular host toolchain.

The most significant distinction is that an embedded toolchain must generate **cross-compiled code**. The compiler, assembler, and linker are configured to emit machine code compatible with the target processor's instruction set (e.g., ARM Cortex-M), rather than the host CPU. This involves selecting the correct ABI and handling architectural details such as endianness, hardware floating-point support, or specialized instruction extensions. In addition to basic code generation, embedded toolchains typically include:

- **Device-Specific Startup Code:** Since embedded systems do not run an operating system by default, the toolchain must provide initialization code that configures the processor state after reset. This includes setting up the stack pointer, initializing memory sections, and defining the interrupt vector table.
- **Linker Scripts for Memory Mapping:** Unlike host systems where memory layout is abstracted by the operating system, embedded applications must explicitly define where code, data, and peripherals reside in memory. The linker script enforces this mapping, ensuring that the binary fits within the device's regions.
- **Runtime Support Libraries:** Embedded toolchains include specialized implementations of standard libraries designed to operate without an operating system. These lightweight libraries provide essential functionality such as math operations or memory management while avoiding features that rely on system calls.
- **Debugging Interfaces:** Many embedded toolchains integrate support for hardware debugging through interfaces like SWD or JTAG. This enables loading binaries onto the target, setting breakpoints, and inspecting registers or memory directly on the device.
- **Output Conversion Utilities:** Since microcontrollers typically require firmware images in raw binary or hex formats, embedded toolchains include utilities to convert the standard elf into target-specific formats such as Intel HEX or Motorola S-Record.

2.3.3 Build Runners

While small embedded projects can be compiled by invoking individual commands manually, this approach quickly becomes impractical as the codebase grows. Modern projects often contain hundreds of source files, complex dependencies, and different build configurations for debugging or optimization.

A **build runner** (or build automation tool) automates this process by defining a set of rules that specify how source files should be transformed into the desired output. At its core, a build runner:

- Tracks dependencies between source files and generated files.
- Determines which parts of the project need to be rebuilt when a file changes.
- Invokes the appropriate compiler, assembler, or linker commands in the correct order.

Build runners operate based on explicit instructions provided through configuration files or scripts.

2.3.4 Build Systems

A **build system** is a higher-level abstraction built on top of build runners. While a build runner executes rules, a build system focuses on generating and managing those rules for complex projects, often in a platform-agnostic way.

Build systems provide several key advantages:

- **Portability:** They enable the same project to be built on different platforms or with different toolchains without rewriting build scripts.
- **Configuration Management:** They support multiple build configurations.
- **Scalability:** They handle projects with multiple modules, libraries, and external dependencies.

In practice, a build system generates low-level build instructions for a runner. For example, it may produce a set of dependency files and rules that a runner can execute efficiently. This separation of concerns allows developers to describe the project structure and relationships at a higher level, without directly managing every compiler or linker invocation.

In the context of embedded systems, build systems are particularly valuable because they can integrate device-specific settings into the build process while remaining flexible enough to support multiple target devices and toolchains.

2.4 Open-CMSIS-Pack

Software compatibility for component re-use has long been a challenge in the microcontroller space, which is much more diverse at the hardware level compared to PCs. Open-CMSIS-Pack removes this complexity, delivering a standard for software component packaging and related foundation tools for validation, distribution, integration, management, and maintenance.

2.4.1 CMSIS-Packs

CMSIS-Packs are software components that contain device specific startup code, peripheral drivers, middleware, and board support packages. They provide a standardized delivery mechanism for software components and enable consistent project configuration across different development environments.

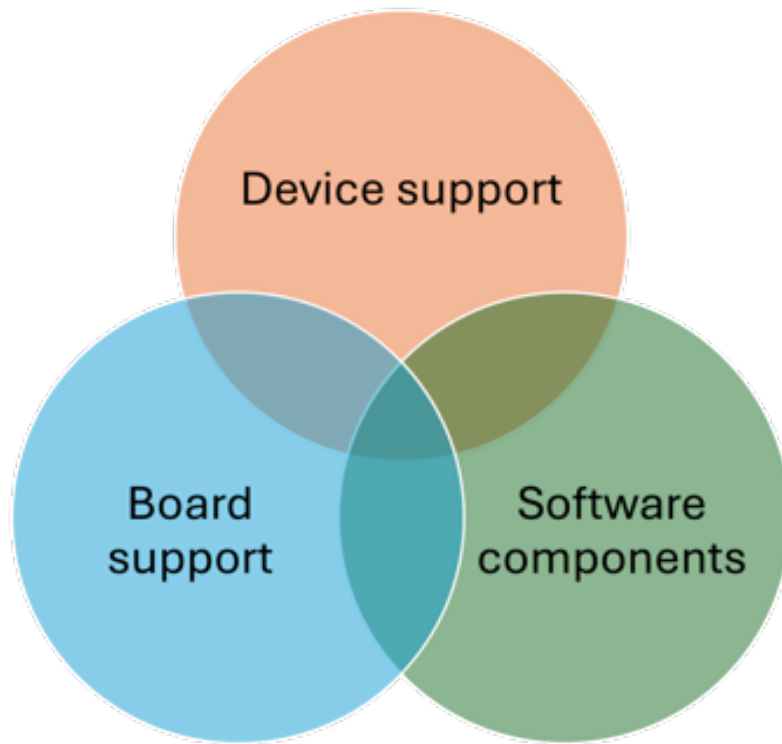


Figure 2.3: Software Packs Types

2.4.2 CMSIS-Pack Format

The CMSIS-Pack format is used to deliver a software package and is aimed to be scalable for future requirements. It provides a management process and supports a tool independent distribution for:

- **Device Support :**

- Information about the processor and it's features.
- C and assembly files for the device startup and access to the memory mapped peripheral registers.
- Parameters, technical information, and data sheets about the device family and the specific devices.
- Device description and available peripherals.
- Memory layout of internal and external RAM and ROM address ranges.
- Flash algorithms for programming the device.
- Debug and trace configurations as well as System View Description files for device specific display of the memory mapped peripheral registers.

- **Board Support :**

- Information about the development board and it's features.
- Parameters, technical information, and data sheets about the board, the mounted microcontroller, and peripheral devices.

- Drivers for on-board peripheral devices
- **Software components :**
 - A collection of source modules, header and configuration files as well as libraries.
 - Documentation of the software, including features and APIs.

2.4.3 Software Components

A software component encapsulates a set of related functions. They can contain C/C++ source files, object code, assembler files, header files, or libraries. The interfaces of software components should be defined with APIs to make them substitutable by other compatible components at design time. CMSIS software components can also refer to multiple interfaces of other software components. This could be also a hardware abstraction layer for a device peripheral. Configuration files contain application specific parameters for a software component. These files are typically copied to the user project workspace; all other files are not modified by the user and can remain in a separate location which avoids that a project workspace is polluted by many source files that should be considered as “black-box” elements by the application programmer.

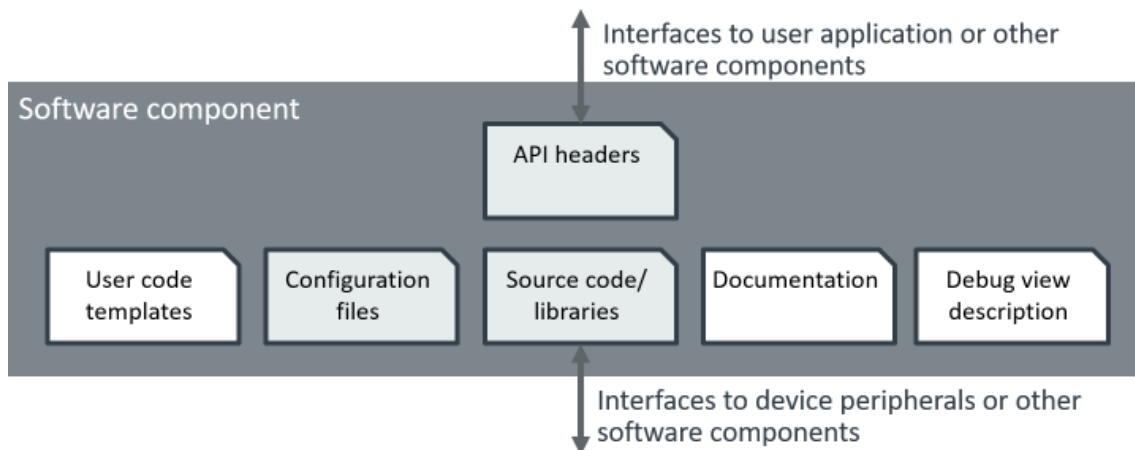


Figure 2.4: Software Component Interface

2.4.3.1 Component classification

A component lists the files that belong to it and that are relevant for a project. The component itself or each individual file may refer to a condition that must resolve to true; if it is false, the component or file is not applicable in the given context. Each component must have the following identifying attributes

- **Component Class (Cclass):** A component class which is a top-level component name, for example CMSIS, Device, File System
- **Component Group (Cgroup):** A component group name, for example CMSIS:RTOS.
- **Component Version (Cversion):** the version number of the software component.

2.4.4 CMSIS Solution Project Structure

As an effort to standardize the embedded software ecosystem, the csolution(CMSIS Solution) project structure came into place, it is a set of configuration files that is meant to describe your overall application, from the workspace, to projects, software layers and default configurations. This approach allows the user to centralize all the configuration of an application.

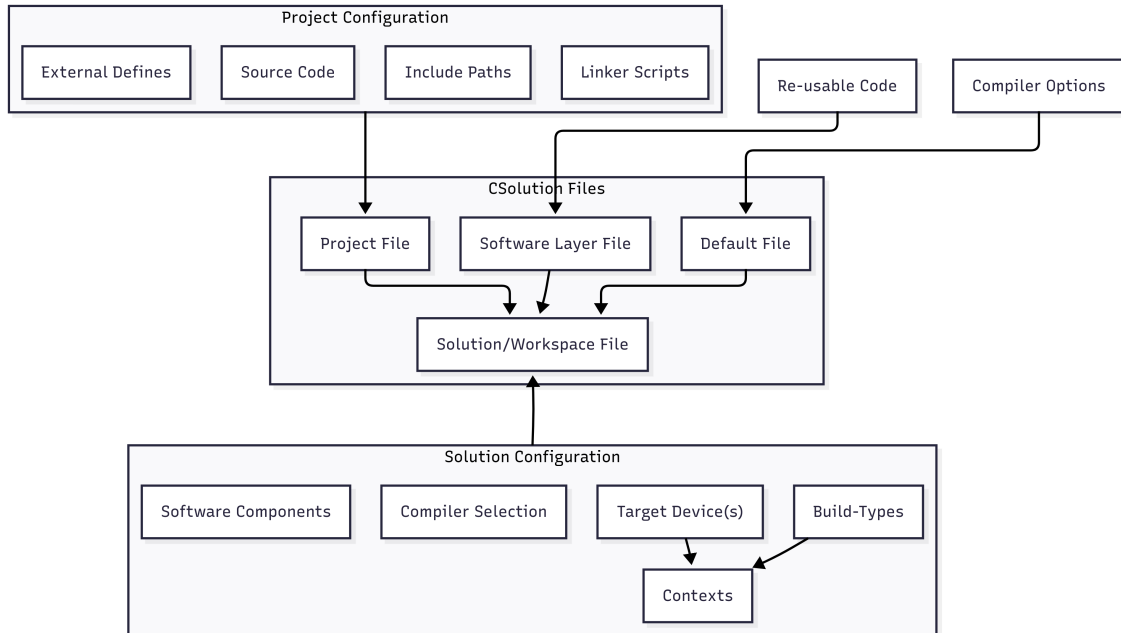


Figure 2.5: CMSIS Solution Project Structure

2.4.4.1 CMSIS solution files

The CMSIS-Toolbox gets its information from the csolution project files, these are YAML configuration files used to describe the application's context:

- **Solution file** A solution is the software view of the complete system. It combines projects that can be generated independently, and specifies the targeted device(s) and build type(s).
- **Project file** The *.cproject.yml file has the content of a single independent build step, it specifies the source files to compile, sets the include directories and the necessary defines.
- **Layer file** Software layers collect source files and software components along with configuration files for reuse in different projects. Software Layers gives projects a better structure and simplifies:
 - Development flows with evaluation boards and production hardware.
 - Evaluation of middleware and hardware modules across different microcontroller boards.
 - Code reuse across projects, i.e. board support for test-case deployment.
 - Test-driven software development on simulation model and hardware.

- **Default Configuration file** The `cdefault.yml` file contains a common set of compiler-specific settings that select reasonable defaults with miscellaneous controls for each compiler.

2.5 Project Generation

When trying to run benchmarks across a wide variety of MCUs, the project structure follows the same pattern, and the dependencies can be expressed in a logical way relating to the device's metadata, therefore, automating the process will prove handy. There are multiple techniques to be used to generate a fully working project from scratch.

2.5.1 Regular Expressions:

The phrase regular expressions, or regexes, is often used to mean the specific, standard textual syntax for representing patterns for matching text. Each character in a regular expression (that is, each character in the string describing its pattern) is either a metacharacter, having a special meaning, or a regular character that has a literal meaning. A regular expression, often called a pattern, specifies a set of strings required for a particular purpose.

2.5.1.1 Operations

Most formalisms provide the following operations to construct regular expressions.

- **Boolean "OR":** A vertical bar separates alternatives. For example, `gray|grey` can match "gray" or "grey".
- **Grouping** Parentheses are used to define the scope and precedence of the operators (among other uses). For example, `gray|grey` and `gr(a|e)y` are equivalent patterns which both describe the set of "gray" or "grey".
- **Quantification** A quantifier after an element (such as a token, character, or group) specifies how many times the preceding element is allowed to repeat. The available quantifications are:
 - **Zero or One:** Denoted by the question mark "?" symbol.
 - **Zero or More:** Denoted by the asterisk "*" (derived from the Kleene star).
 - **One or More:** Denoted by the plus symbol "+".
 - **N or More:** Denoted by the expression : "{N,}" where N is a positive integer.
 - **N or Less:** Denoted by the expression : "{,N}" where N is a positive integer.
 - **Between N and M times:** Denoted by the expression: "{N, M}" where N and M are positive integers such that $N < M$.
- **Wildcard:** Denoted by the dot "." it matches any single character

2.5.2 Templates & File Manipulation

File generation templates are pre-defined structures or blueprints used to create new files with standardized content and formatting.

2.5.2.1 Template Definition

A template file is created containing boilerplate text, placeholders for dynamic content (variables), and sometimes logic for conditional generation.

2.5.2.2 Parameterization

The template defines parameters that can be filled in by the user or a program when generating a new file. These parameters can be single-valued, arrays or maps containing relevant information.

2.5.2.3 Template Execution

When a new file is needed, the template is selected, and the required parameters are provided. A generation engine then processes the template, replacing placeholders with the provided values and executing any embedded logic to produce the final file.

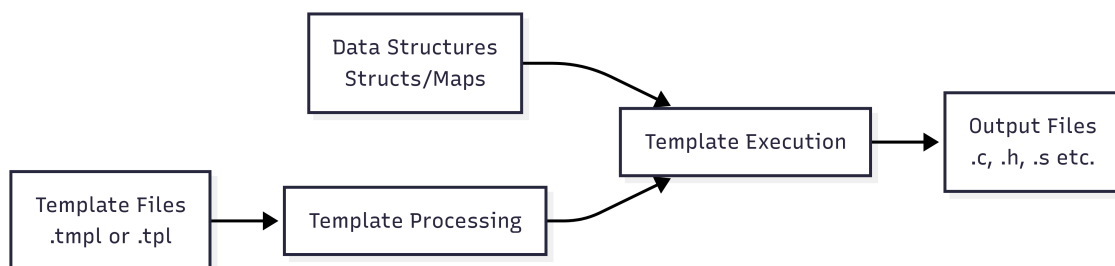


Figure 2.6: Templating Process

Conclusion

This chapter has provided an overview of the key theoretical concepts and technologies relevant to the project. We began by exploring the STM32 microcontroller ecosystem, including the various series and ARM Cortex-M cores that form the foundation of these devices. The STM32Cube firmware package was introduced as a comprehensive software suite for STM32 development.

We then examined the principles of benchmarking, with a particular focus on the CoreMark benchmark and its application to microcontrollers. The C/C++ build process for embedded systems was detailed, highlighting the unique requirements and toolchains needed for cross-compilation.

Finally, we discussed methods for automating project generation, including the use of regular expressions and file templates. These concepts and tools form the theoretical and practical basis for the work conducted during this internship, setting the stage for the implementation and results that will be presented in subsequent chapters.

OBJECTIVES SPECIFICATION AND WORK ENVIRONMENT

Contents

1	Project Specification	25
2	Structural Decisions	26
3	Use-Case Diagram	28
4	Hardware Resources	30
5	Software Resources	33

Introduction

This chapter will first define the project's functional and non-functional requirements, then we will discuss the structural decisions, going over the reasons behind these choices. Finally we will specify the hardware and software resources necessary for this project.

3.1 Project Specification

Requirements analysis is a fundamental phase in every project realization process. It is based on the study of the project's features as well as the constraints. In this section, we will cover both the functional and non-functional requirements.

3.1.1 Functional Requirements

The goal of the project is to generate a running CoreMark project on STM32 devices using CMSIS-Toolbox in an automated way. Within this context, this means guaranteeing the following concepts:

Table 3.1: Functional Requirements for CMSIS-Toolbox based automated benchmarking

Requirements	Description
Automation	The system must automatically generate a CoreMark benchmarking project for STM32 devices without requiring manual configuration.
Device Adaptability	The generated project must adapt to different STM32 families and configurations.
CoreMark Integration	The CoreMark benchmark must be integrated and ready to run immediately on the target hardware.
Result Reporting	The system must provide a standardized mechanism for reporting benchmark results.
Multi-Toolchain Support	The project must support multiple compilers such as GCC, ARM Compiler, and IAR.

3.1.2 Non-Functional Requirements

Below are the non-functional requirements or the constraints that the functional expectations must abide by:

- **Performance Requirements:**
 - **Low Overhead** The project must not add significant, if any overhead besides the CoreMark runtime, this insures the integrity of the benchmarking results.
 - **Compiler Optimizations** The project must provide the most optimal compiler configuration to ensure the maximum performance results.

- **Reliability Requirements:**

- **Correctness** The project must be functional and able to run without manual fixes.
- **Consistency** The project must produce consistent results amongst STM32 devices.
- **Error Handling** The project must fail gracefully and generate meaningful error messages.

- **Usability Requirements:**

- **Ease of use** The generation process must be simple and intuitive, the project structure must be clear and consistent.
- **Standardized Output** The results must be presented in a clear format to facilitate further automation and data collection.

3.2 Structural Decisions

Before delving further into the implementation details, it is crucial to understand the rationale behind the structural decisions chosen, for both the generation tool and the CoreMark project. This sections elucidates the reasons behind the choices and their relevance to the project's objectives.

3.2.1 CoreMark Project

The CoreMark project is the end result of the generation tool, it is meant to be runnable out of the box with the 3 main compilers (GCC, ARM, and IAR).

Ensuring the cross-toolchain support is done by providing the necessary compiler specific directives:

- **Linker Scripts** Each compiler has their own unique format for linker scripts:
 - **GCC:** GCC uses the .ld file extension and the LDScript syntax for the linking phase, it is structured in a declarative way to generate regions, sections and describe behavior within elements as well as exporting symbols.
 - **IAR:** IAR uses the .icf file extension as well as proprietary syntax for the linking phase, it allows the user to simply declare regions and optionally sections and takes care of the rest.
 - **ARM:** ARM compiler uses the .sct file extension, you only have to declare memory regions and the rest is handled either in C code or by the compiler directly.
- **Startup Files** Within the provided versions in the STM32CubeFirmware, startup files are toolchain dependent, the main reason is that they depend on the linker script's specific symbols.
- **Compiler Options** Each compiler has their own set of compiler flags to be used.

The table 3.2 provides an overview of the architecture of the CoreMark project.

Table 3.2: CoreMark Project Structure

Element	Description
User Code	This component provides the definition for the user application's entry point as well as the UART peripheral configuration.
Driver	This component includes the necessary HAL drivers to enable the UART peripheral.
Modifiable Files	This component contains the toolchain specific source and header files to modify if the user wants to modify system configurations such as linker scripts and startup files.
CoreMark	This component contains the source code of the CoreMark benchmark to be invoked within the usercode's entry point.
coremark.csolution.yml	This is the workspace file for CMSIS-Toolbox to have knowledge of the project, it defines the DFP, ARM's CORE component, alongside the target device, build types and compiler selection.
coremark.cproject.yml	This is the project description file, it contains the source files to be compiled, user defines for UART functionality and the include paths.
cdefault.yml	This is the file that contains all the compiler options, in our context, the configuration is meant to provide the most performance-oriented setup.

3.2.2 Project Generation Tool

The architecture of the CoreMark project is quite complex and contains multiple pieces that must be either generated dynamically or fetched from different resources.

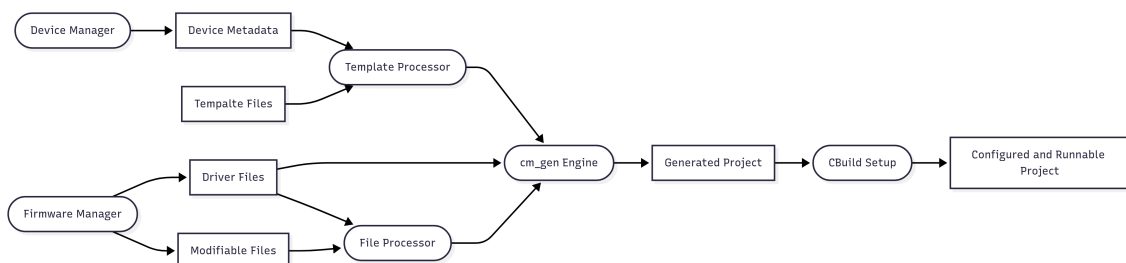


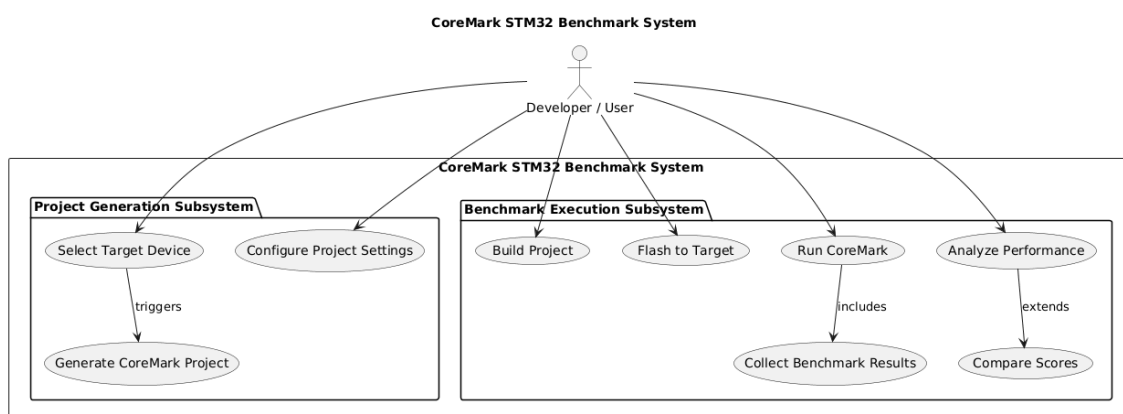
Figure 3.1: Project Generation Tool Architecture

The figure 3.1 shows the multiple modules within the project generation tool.

Table 3.3: Project Generation Tool Structure

Module	Description
Device Manager	Extracts the device metadata
Firmware Manager	Manages the device's firmware pack and extracting the necessary files
Template Processor	Generates files from the template files and the gathered data
File Processor	Modifies firmware files that cannot be templated to accomodate for our project's context
cm_gen Engine	Orchestrates and runs the different modules

3.3 Use-Case Diagram

**Figure 3.2:** Automated CoreMark Benchmarking Use-Case Diagram

The Use-Case diagram in Figure 3.2 illustrates the interactions between a user, the project generation tool and the project's execution space. Refer to Table 3.4 for an in-depth explanation:

Table 3.4: Automated CoreMark Benchmarking

Use Case	Relevance
Project Generation Subsystem	
Select Target Device	The developer selects the STM32 device or family for which the CoreMark project should be generated. This ensures that the automation tool generates the correct configuration files and CMSIS device support.
Configure Project Settings	The developer configures CoreMark-specific parameters such as iteration counts, seed values, and compiler optimization flags before building and running the benchmark.

Continued on next page

Table 3.4 – Continued from previous page

Use Case	Relevance
Generate CoreMark Project	The automation tool generates a complete project structure, including the csolution files and necessary source code, ready to be built and deployed for the selected STM32 target.
CoreMark Project Subsystem	
Build Project	The CoreMark project is compiled using the CMSIS build system and toolchain to produce a firmware image for the target hardware.
Flash Target	The compiled binary is flashed to the STM32 device to prepare it for execution.
Run CoreMark	The benchmark executes on the STM32 target, performing operations such as list processing, matrix manipulation, and state machine handling to measure CPU performance.
Collect CoreMark Results	During execution, CoreMark outputs the cycle count, execution time, and final score.
Analyze Performance	The developer interprets the CoreMark/MHz score and other performance metrics to determine how well the device performs within the specified context.
Compare Scores	Performance results are compared across different devices, toolchains, or configurations to guide hardware selection or optimization decisions.

Work Environment

In this section, we outline the hardware and software resources utilized during the internship project that were essential for the development testing, and validation.

3.4 Hardware Resources

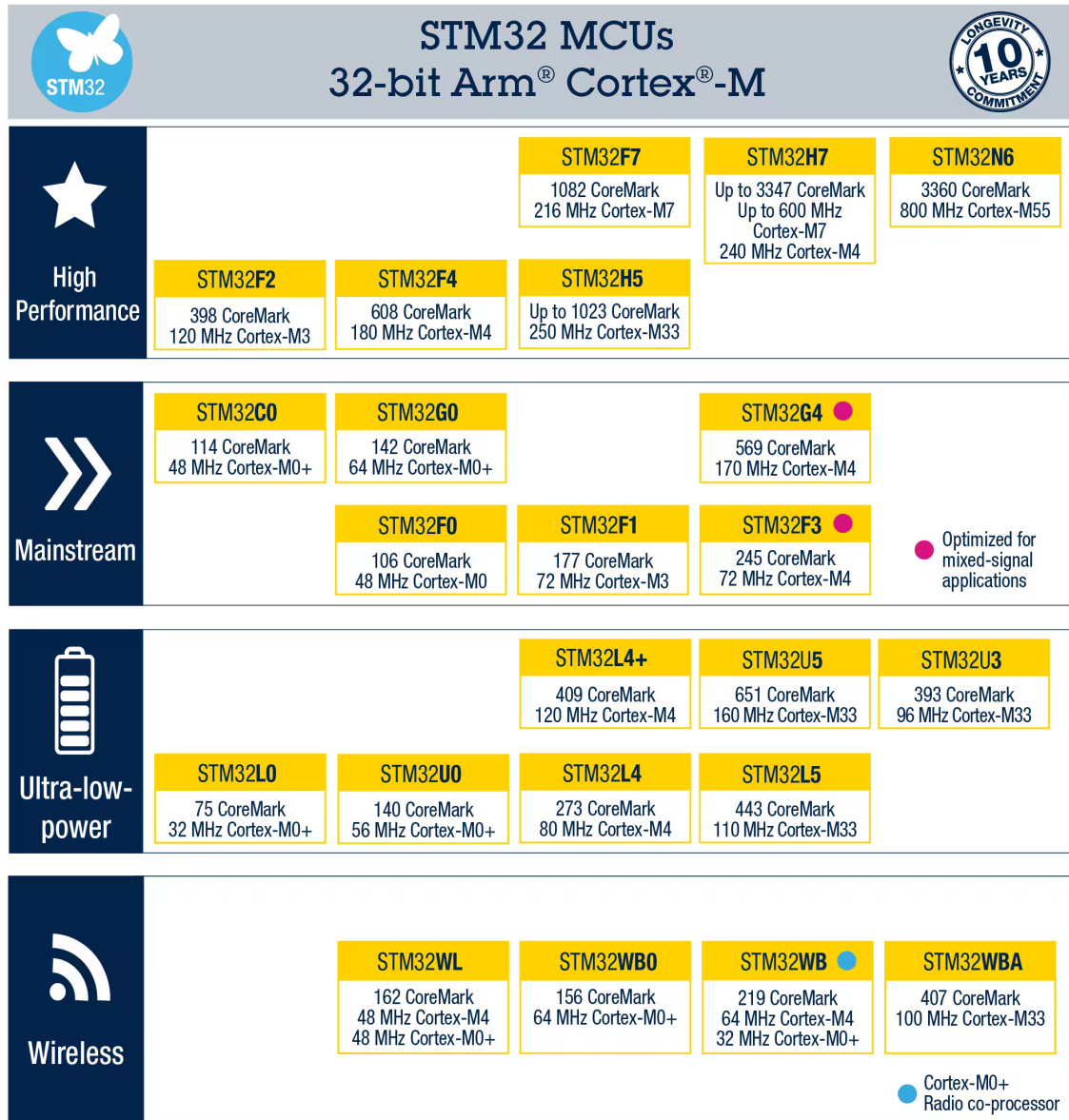


Figure 3.3: STM32 Microcontroller Portfolio

The project is meant to work on all STM32 MCUs, therefore as a base unit, the validation was done on a single board from all STM32 MCU families. Table 3.5 describes all the available families:

Table 3.5: STM32 Microcontroller Families

Family	Core	Maximum Frequency(MHz)	Read-Only Memory	Random Access Memory	Additional Features
STM32C0	Arm Cortex-M0+	48MHZ	16-256KB	6-36KB	-
STM32F0	Arm Cortex-M0	48MHz	16-256KB	4-32KB	-
STM32F1	Arm Cortex-M3	72MHz	16KB-1MB	4-96KB	-
STM32F2	Arm Cortex-M3	120MHz	128KB-1MB	64KB-128KB	ART Accelerator
STM32F3	Arm Cortex-M4	72MHz	32-512KB	16-80KB	CCM-SRAM and FPU/DSP instructions
STM32F4	Arm Cortex-M4	180MHz	64KB-2MB	32-384KB	Chrom-ART Accelerator
STM32F7	Arm Cortex-M7	216MHz	64K-2MB	256-512KB	L1 Data and Instruction Caches, TCM, Read-While-write Flash capabilities
STM32G0	Arm Cortex-M0+	64MHz	16-512KB	8-144KB	
STM32G4	Arm Cortex-M4	170MHz	32-512KB	32-128KB	CCM-SRAM and FPU/DSP instructions
STM32H5	Arm Cortex-M33	250MHz	128KB-2MB	32-640KB	Integrated Instruction and Data Cache peripherals

Continued on next page

Table 3.5 – Continued from previous page

Family	Core	Maximum Frequency	Read-Only Memory	Random Access Memory	Additional Features
STM32H7	Arm Cortex-M7	600MHz	64KB-2MB	536KB-1.4MB	L1 Data and Instruction Cache, TCM, Chrome-ART and NeoChrom Accelerators.
STM32L0	Arm Cortex-M0+	32MHz	8-192KB	2-20KB	Up to 6KB of EEPROM
STM32L1	Arm Cortex-M3	32MHz	32-512KB	4-80KB	Up to 16KB of EEPROM
STM32L4	Arm Cortex-M4	80MHz	64KB-1MB	40-320KB	Chrom-ART Accelerator
STM32L4+	Arm Cortex-M4	120MHz	512KB-2MB	320-640KB	Chrom-ART Accelerator
STM32L5	Arm Cortex-M33	110MHz	256-512MB	256KB	ART Accelerator
STM32N6	Arm Cortex-M55	800MHz	0	4.2MB	Neural-ART Accelerator(Up to 1GHz Clock Speed)
STM32U0	Arm Cortex-M0+	56MHz	16-256KB	12-40KB	-
STM32U3	Arm Cortex-M33	96MHz	512KB-1MB	256KB	Integrated Instruction and Data Cache peripherals

Continued on next page

Table 3.5 – Continued from previous page

Family	Core	Maximum Frequency	Read-Only Memory	Random Access Memory	Additional Features
STM32U5	Arm Cortex-M33	160MHz	128-4MB	274KB-3MB	Integrated Instruction and Data Cache peripherals
STM32WB	Arm Cortex-M4	64MHz	256KB-1MB	48-256KB	-
STM32WB0	Arm Cortex-M0+	64MHz	192-512KB	24-64KB	-
STM32WBA	Arm Cortex-M33	100MHz	512KB-2MB	64-512KB	Integrated Instruction and Data Cache peripherals
STM32WL	Arm Cortex-M4	64MHz	64-256KB	8-64KB	-

3.5 Software Resources

The following software tools were used for the development, debugging and testing of the project:

3.5.1 Programming Languages

- **C:** C is a general-purpose programming language. By design, it gives the programmer relatively direct access to the features of the typical CPU architecture, customized for the target instruction set. It has been and continues to be used to implement operating systems (especially kernels), device drivers, protocol stacks, and embedded applications.
- **Go:** Go (or Golang) is a high-level general purpose programming language that is statically typed and compiled. It is known for the simplicity of its syntax and the efficiency of development that it enables by the inclusion of a large standard library supplying many needs for common projects.

3.5.2 Integrated Development Environments and Text Editors

3.5.2.1 Visual Studio Code

Visual Studio Code (VS Code) is an extensible code editor developed by Microsoft for Windows, Linux, macOS and web browsers. Features include support for debugging, syntax highlighting, intelligent code completion,

snippets, code refactoring, and embedded version control with Git. Users can install extensions that add functionality. Table 3.6 goes over the most impactful extensions during the development of the project.

Table 3.6: Relevant VSCode Extensions

Extension	Description
Arm CMSIS Solution	The Arm CMSIS Solution extension is a graphical user interface for csolution projects that use the CMSIS-Toolbox. The extension supports microcontroller devices that incorporate Arm Cortex®-M processors and works with various C/C++ compilers and debuggers.
Arm CMSIS Debugger	The Arm CMSIS Debugger extension pack is a comprehensive debug platform for Arm Cortex-M processor-based devices that uses the GDB/MI protocol. It supports single and multi-core processor systems, it includes pyOCD for target connection and image download, as well as GDB for core debugging features.
Serial Monitor	The Serial Monitor extension provides a serial monitor to view output from as well as send messages to serial ports. This is often useful when testing or debugging programs on embedded devices.
ELFInsight	ElfInsight is an extension to analyze ARM Cortex-M ELF files, it offers a streamlined interface to view symbol tables, inspect memory usage and visualize function call graph.

3.5.3 Compiler Toolchains

3.5.3.1 GCC

The GCC compiler is an open-source C/C++ compiler, and is part of the GNU utilities, ARM provides their own implementation, prefixed with "arm-none-eabi" and can target most ARM architectures.

3.5.3.2 ARM Compiler v6

ARM has their own closed source compiler based on CLANG and LLVM technologies.

3.5.3.3 IAR Toolchain

IAR Systems is a Swedish computer software company that offers development tools for embedded systems. They are known for having a comprehensive development suite and an optimized compiler based on GCC.

3.5.4 Development Tools

3.5.4.1 CMake

CMake is a free, open-source, cross-platform, software development tool for building applications via compiler-independent instructions. It also can automate testing, packaging and installation. It runs on a variety of platforms and supports many programming languages. As a meta-build tool, CMake configures native build tools which in turn build the codebase. CMake generates configuration files for other build tools based on CMake-specific configuration files. The other tools are responsible for more directly building; using the generated files. A single set of CMake-specific configuration files can be used to build a codebase using the native build tools of multiple platforms.

3.5.4.2 Ninja

Ninja is a small build runner with a focus on speed. It differs from other build systems in two major respects: it is designed to have its input files generated by a higher-level build system, and it is designed to run builds as fast as possible.

3.5.4.3 CMSIS-Toolbox

The CMSIS-Toolbox provides command-line tools for project creation and build of embedded applications utilizing CMSIS-Packs. It supports multiple compilation tools. It also helps with software pack creation, maintenance, and distribution utilizing the CMSIS-Pack format.

Overall Workflow

The CMSIS-Toolbox includes the following tools for the creation of embedded applications:

- **Pack Manager (cpackget):** install and manage software packs in the host development environment.
- **Project Manager (csolution):** create build information for embedded applications that consist of one or more related projects. It performs the following operations:
 - **In the Project Area:** Generate build information files *.cbuild-idx.yml and *.cbuild.yml with all relevant project information for the build process.
 - **In the RTE Directory:**
 - * Generate for each context the RTE_components.h file and pre-include files from the software pack (*.pdsc) metadata.
 - * Copy the configuration files from selected software components and provide PLM information.

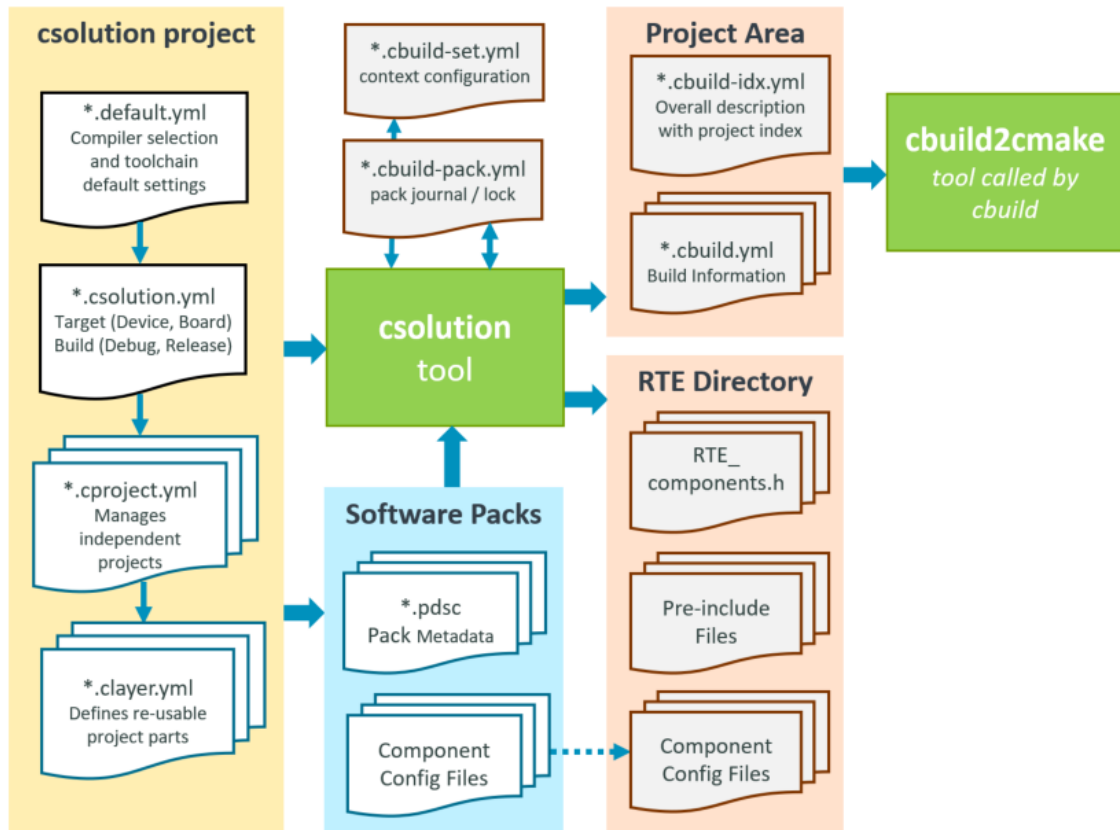
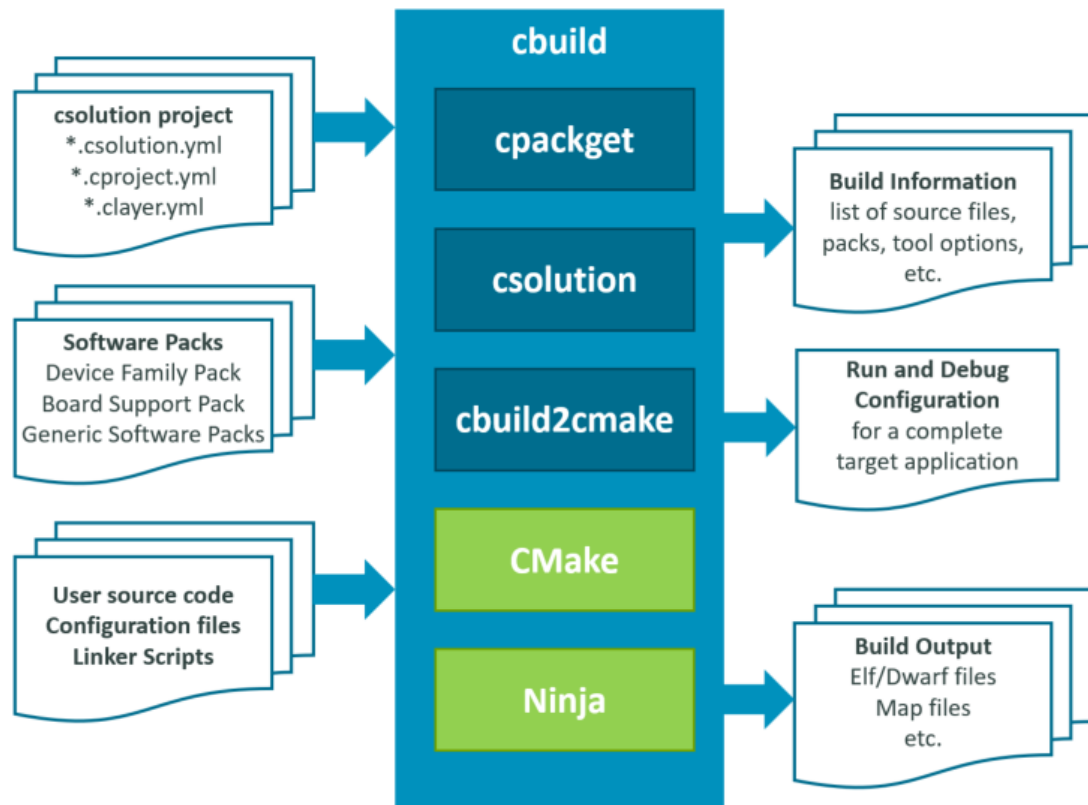


Figure 3.4: Csolution Operation

- **Build Invocation (cbuild):** orchestrate the build steps utilizing CMSIS tools and a CMake compilation process. It has two overall modes :
 - **build mode:** generates the application and is the default command.
 - * When option `-packs` is used, it downloads missing software packs using `cpackget`.
 - * It calls `csolution` to process the `<name>.csolution.yml` project. The output are build information files with all relevant project information for the build process.
 - **setup mode:** generates the setup information for an IDE to populate dialogues, IntelliSense, and project outline views.
 - * Check YML file syntax against schema for all files specified by `*.csolution.yml`.
 - * Check the correctness of all files specified by `*.csolution.yml`.

**Figure 3.5:** Cbuild Workflow

Conclusion

The work environment for this internship includes both hardware and software resources that are essential for the successful implementation of the project.

SOLUTION IMPLEMENTATION

Contents

1	Project Development Overview	39
2	CoreMark Project Architecture	45
3	Project Generation Tool	52

Introduction

In this chapter, we provide an in-depth explanation of the technical aspect of our project, we will go over the project's evolution over its lifetime, the architecture of the CoreMark project and the generation tool's inner workings.

4.1 Project Development Overview

This section describes how the project evolved in a chronological order, alternating between research, development, testing and analysis.

4.1.1 Porting CoreMark On STM32 Devices

The first step during the internship was to get familiar with CoreMark, studying how it works and figuring out the necessary configurations to get it to work on embedded systems, particularly STM32 devices. The structure of CoreMark's source code is done in a way such that all platform specific dependencies are located in a source header file pair called `core_portme.c` and `core_portme.h` respectively. Within these files, we find the necessary changes and implementations needed to run the project.

4.1.1.1 Clock Source

CoreMark needs the user to implement a timing function in order to provide accurate measures of the CPU's performance.

In embedded systems, Timers are the most efficient way of time measurement. A timer relies on a tick register that decrements periodically, and possibly resets to its maximum value when it reaches 0.

ARM provides a 24-bit system timer SysTick integrated in Cortex-M processor cores. Manipulating this timer is done via 4 registers :

- **SYST_CSR** The SysTick Control and Status register enables the SysTick features, such as specifying whether the clock source is external or internal, enabling exceptions and enabling the timer
- **SYST_RVR** The SysTick Reload Value register specifies the start value of the timer, this is the value to be loaded into SYST_CVR every time its value reaches 0.
- **SYST_CVR** The SysTick Current Value register contains the current value of the SysTick counter.
- **SYST_CALIB** The SysTick Calibration Value register indicates the SysTick calibration properties, it is optional and not required to be implemented.

Figures 4.1 and 4.2 below show the implementation details of the start and stop functions for CoreMark.

```

#define SYSTICK_UF 0xFFFFF
void start_time(void)
{
    start_time_val = 0;
    SysTick->LOAD = SYSTICK_UF;
    SysTick->VAL = 0;
    SysTick->CTRL |= 1;
    uwTick = 0;
}

```

Figure 4.1: CoreMark Timing Start Function

```

void stop_time(void)
{
    stop_time_val = (uint64_t) uwTick * SYSTICK_UF + (uint64_t) SysTick->VAL;
}

```

Figure 4.2: CoreMark Timing End Function

When working across the entire STM32 MCU catalog, some families can have a default base clock speed up to 128MHz, therefore, we will be using the maximum value allowed for the counter(2^{24} Ticks) to reduce the number of interrupts fired during the execution of the benchmark.

Enabling the SysTick timer is done by setting the first bit(index 0) of the SYST_CSR register to 1.

To keep track of how many times the SysTick timer underflows, we utilize the 32-bit unsigned integer variable provided by ST's HAL library uwTick, we implement the SysTick interrupt handler to increment its value by 1 everytime it fires.

With this method, we now can get the total number of ticks elapsed during the execution of our program with this formula :

$$TotalTicks = uwTick * SysTickReloadValue + SysTickCurrentValue$$

Subsequently, the total time elapsed in seconds would equate to:

$$Total\ Time\ in\ Seconds = TotalTicks / ProcessorFrequency$$

4.1.1.2 Result Reporting

To report CoreMark's results, the implementation of the benchmark relies on providing a definition to a function called ee_printf, on high-end systems making use of operating systems, we can make use of the default implementation of the libc standard library's printf function.

However, in our case, we do not have a defined standard output interface, so, we will be relying on STM32 MCUs' peripherals to share the data with the user.

Since benchmarks will mainly be done on boards, an important feature available on them is that the

integrated ST-Link provides a VCP connected to one of the MCU's UART peripherals, allowing us to emulate the serial port over a USB connection. Therefore, we design our `ee_printf` function, to send the data over the specified UART peripheral. Figures 4.3 and 4.4 below show the implementation details of the printing process.

```
int ee_printf(const char *fmt, ...)
{
    char    buf[1024], *p;
    va_list args;
    int     n = 0;

    va_start(args, fmt);
    ee_vsprintf(buf, fmt, args);
    va_end(args);
    p = buf;
    while (*p)
    {
        send_char(*p);
        n++;
        p++;
    }

    return n;
}
```

Figure 4.3: CoreMark `ee_printf` Function Definition

```
void send_char(char c){
    HAL_UART_Transmit(&huart2, (uint8_t*) &c, sizeof(c), HAL_MAX_DELAY);
}
```

Figure 4.4: Function to send a single character over UART

```
Running CoreMark at 250MHz
2K performance run parameters for coremark.
CoreMark Size      : 666
Total ticks        : 537145951
Total time (secs): 15.267116
Iterations/Sec     : 720.502817
Iterations         : 11000
Compiler version   : GCC14.2.1 20241119
Compiler flags     : -O3
Memory location    : STACK
seedcrc           : 0xe9f5
[0]crclist        : 0xe714
[0]crcmatrix      : 0x1fd7
[0]crcstate       : 0x8e3a
[0]crcfinal       : 0x33ff
Correct operation validated. See README.md for run and reporting rules.
CoreMark 1.0 : 720.502817 / GCC14.2.1 20241119 -O3 / STACK
```

Figure 4.5: Sample CoreMark Results Transmitted over UART

4.1.2 Running CoreMark Using Csolution Project Structure

After successfully porting CoreMark to work on STM32 devices, the next step is to leverage the Csolution project structure within our CoreMark project.

Following on the information provided in Chapter 2, Section 2.4.4, the process of utilizing csolution project structure consists of creating the configuration files within our application.

The main points to get the project working were:

- Providing the source files to compile.
- Providing the include paths.
- Providing the target device and build types.
- Providing the linker scripts for each compiler.

Section 4.2.4 goes into further detail on the content of the files and the logical reasoning behind it.

4.1.3 Project Structure Definition

The csolution project structure offers multiple ways to embed software within your application, for example, since the CoreMark source files are static, we could have defined CoreMark as a software layer to be callable by any project we want to create. However, the most optimal approach is to compile all the source files into a single image, as providing the benchmark as a position independent code might result in sub-optimal performance, especially with memory placements of critical sections of the code. Even with this approach, we can still logically divide our files into groups and have conditional compilation by leveraging features of the CMSIS environment. Figure 4.6 Describes the overall project structure, the Subecetion 4.2 dives into the details of the architecture.

```
coremark/  
drivers/  
modifiable/  
user_code/  
cdefault.yml  
coremark.cproject.yml  
coremark.csolution.yml
```

Figure 4.6: Overview of the CoreMark project structure

4.1.4 Automation Analysis

At this point, with the overall architecture defined and the key points to get a project running on an STM32 device clear and made. The generation process still is manual, and we have multiple components of the system

to keep in mind in order to get to a working state, such a process is error prone and isn't much different from manually setting up the project with the traditional approach via build systems such as CMake. The next step in the project was to analyze what we have done in the prior steps, and identify the processes we can automate.

Table 4.1: Automatable Patterns in CoreMark Project Generation

Automation Aspect	Description
Device-Specific Naming	Multiple definitions, symbols, and file references follow a strict naming convention based on the target device. Examples include the Device Family Pack (DFP) name and device headers. These can be programmatically generated based on the target STM32 device identifier.
UART Configuration	Source files include patterns tied to the selected UART peripheral (e.g., HAL handle names, initialization functions). Automating UART setup ensures consistent configuration across projects.
Manual HAL/LL Driver Integration	Without STM32CubeMX, HAL and LL drivers must currently be copied manually. The structure of the STM32CubeFirmware directory and the online availability of firmware packages make it possible to automatically locate, download (if missing), and include only the minimal required modules for the target device.
Project Structure and Templates	Core files such as startup code, linker scripts, and benchmark integration follow predictable patterns. These can be generated dynamically based on a predefined template, reducing repetitive manual work.
Dependency Management	Ensuring the presence of external resources (e.g., DFP, STM32CubeFirmware) can be automated by checking the environment and fetching missing components, minimizing setup errors.

After carefully reviewing the aspects provided in Table 4.1, we can also derive the minimal required information to enable these aspects.

- **Device Name:** The device name allows us to extract the majority of the data we need, from the device family, ARM Cortex-M core, even the FLASH memory size and pin count are available.

- **UART Instance:** While this data can be extracted from the device name, we have opted for requiring this information to ensure consistency when taking devices yet to be released into consideration.

4.1.5 Fully Automated Generation

After identifying the key points and the minimum required information to automate, we will then proceed to create the project generation tool labeled "cm_gen".

cm_gen is a CLI tool written in Golang aimed to generate a fully functioning CoreMark project on STM32 devices using csolution project structure and works on all compilers out of the box. Figure 4.7 describes the workflow of the application from a high-level perspective, further details are provided in Section 4.3

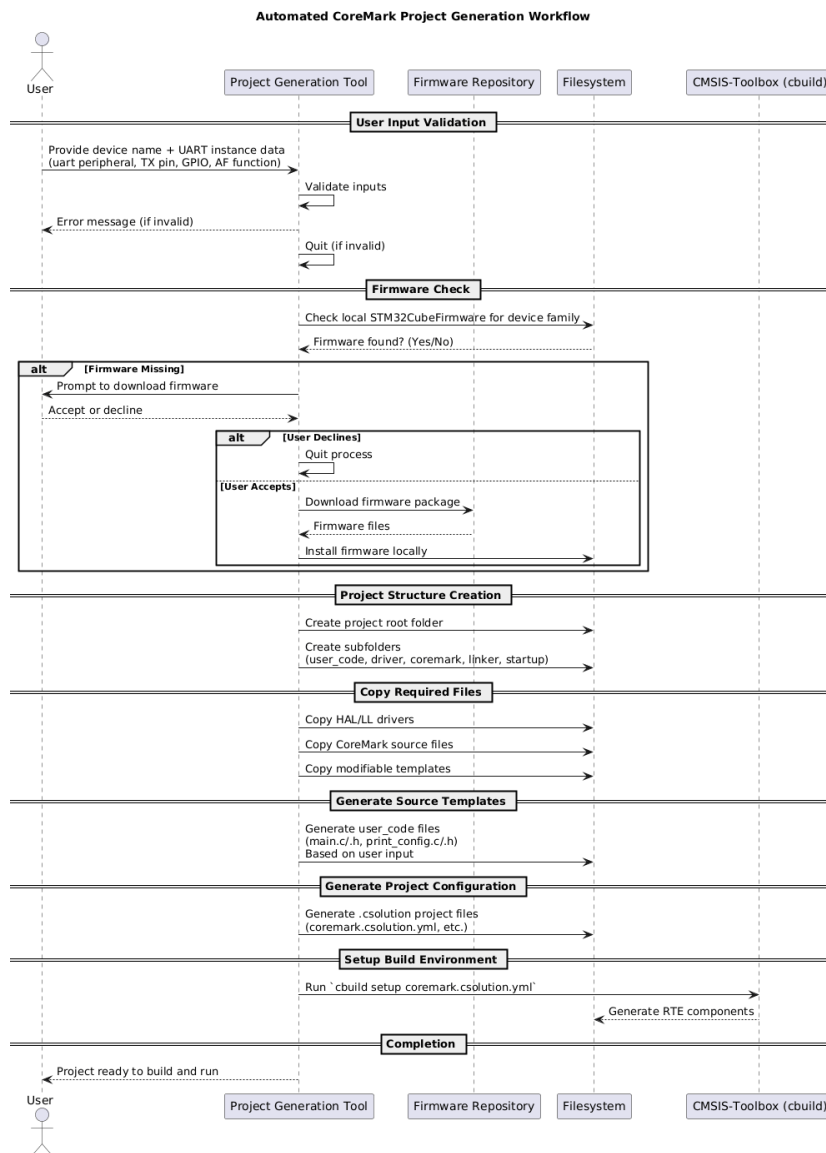


Figure 4.7: cm_gen Sequence Diagram

4.1.6 Project Validation

Validating the summer internship project consists of validating that the csolution based project structure works on STM32 devices.

This was done by setting up and running the project on a device from every single STM32 MCU family, the results were then stored in an Microsoft Excel spreadsheet, compared to results obtained manually and in other work environments. After making sure everything works as intended, the spreadsheet became a reference and a baseline for CoreMark results across all STM32 MCU families and compiler toolchains.

4.2 CoreMark Project Architecture

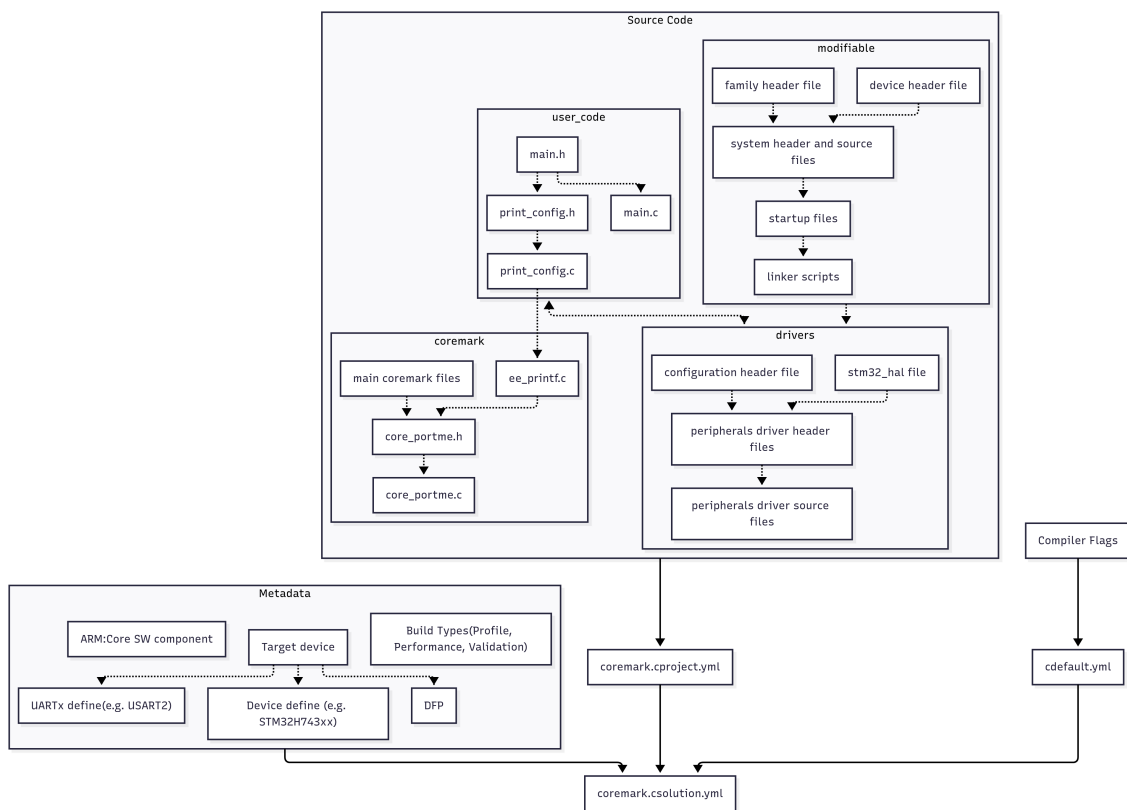


Figure 4.8: CoreMark Project Architecture Overview

Figure 4.8 shows the overall architecture of the project, source files are divided into groups to highlight modularity. We can observe the extra information in the form of metadata and compiler flags we need to provide for the CMSIS-Toolbox to work as well.

The following subsections dive into the details of each component of the project.

4.2.1 User Code

The user code component is made of two pairs of source/header files `main` and `print_config`.

- **Main:** The main header file imports the family's HAL header file. The main file contains the user application's entry point as well as the redefinition of the SysTick_Handler function for reasons mentioned in the Subsection 4.1.1.1, the entry point executes the following sequence then enters an empty infinite loop:
 - Calls the HAL initializer function
 - Calls the print configuration function
 - Calls the CoreMark benchmark's entry function
- **Print Configuration:** The header file declares the exported functions to be called, the source file provides the implementation, these functions are the UART peripheral initialization and the implementation of the send_char function show in Figure 4.4.

4.2.2 Modifiable Files

The modifiable group is a set of files meant to be altered by the user in case they want to change the behaviour and state of the system when executing CoreMark.

- **Linker Scripts:** The linker script defines the memory layout of the application, in the CoreMark application, the default linker scripts are a static sample that offers the bare minimum to get the project running:
 - **Entry Point:** The execution's entry point is different from the user application's entry point, as there's initial setup to be done before entering the user application, this symbol is provided to the linker script in order to know where to start executing.
 - **Memory Regions:** The linker script defines two main regions, **ROM** and **RAM**, the ROM region, is a 64KB region starting at address 0x08000000, it is meant to hold all text symbols, where as RAM is a 12KB region starting at address 0x20000000, it contains all the data symbols and non initialized variables.
 - **Sections:** For the IAR and ARM compilers, regions are equivalent to sections, you only have to define the type of data provided to each region based on permissions (**Read**, **Write**, **Execute**), however, for gcc, sections are the containers of symbols, we then specify in which memory region each section resides. Therefore we define the following sections:
 - * **Text:** This section contains the instructions of the program. It is typically placed in the ROM memory region and is marked with execute permissions.

- * **Data:** This section stores initialized data, such as global or static variables with predefined values. It is placed in the RAM memory region and requires read and write permissions to allow the program to access and modify these variables during execution.
- * **BSS:** The BSS section contains uninitialized data, similar to the data section, the difference is that the startup file zeroes out this section by default.
- **Startup Files:** The startup file defines the interrupt vector table, defining a default weak implementation for each interruption, also, it defines the entry point function, typically named **Reset_Handler**, this function starts by copying the appropriate data from ROM to RAM, and zeroes out the BSS section, afterwards it calls the System Initialization function (`System_Init`), `libc`'s initialization function and finally jumps into the user application's code.
- **System Files:** The system header/source files pair implements core system-level initialization functions.
 - Configure the clock tree and default peripheral states at reset.
 - Provide global variables like `SystemCoreClock` used by CMSIS and HAL drivers.
 - Setup memory banks and the Vector table location and offset addresses.

4.2.3 Static Folders

4.2.3.1 Driver

The driver group contains the necessary files from the HAL/LL libraries to get the RCC and UART peripherals working, the modules used are :

Table 4.2: HAL Modules Used in the CoreMark Project

Module	Description
RCC	Provides functions to configure and control the system clock, oscillators, and peripheral clock sources.
FLASH	Manages FLASH memory operations such as wait states configuration and access control for optimal performance.
PWR	Handles power control features, including voltage scaling and low-power modes.
GPIO	Configures and controls general-purpose input/output pins, including alternate functions for peripherals.
UART	Provides initialization and data transfer functions for UART communication.

4.2.3.2 CoreMark

The coremark folder contains coremark's source code and ee_printf implementation as describe in Table 4.3.

Table 4.3: CoreMark Source Files

File	Description
core_main.c	Contains the main CoreMark benchmark logic and workload orchestration.
core_state.c	Implements the state machine used by the CoreMark benchmark algorithms.
core_util.c	Provides utility functions such as CRC calculations and benchmark support routines.
core_matrix.c	Implements the matrix manipulation workload used by CoreMark.
core_list_join.c	Implements the linked-list joining workload used to stress pointer-chasing and memory allocation patterns.
core_portme.c	Bare-metal porting layer connecting CoreMark to the STM32 platform, including timing and initialization.
core_portme.h	Header file for platform-specific definitions and configuration options used in <code>core_portme.c</code> .
ee_printf.c	Minimal implementation of <code>ee_printf</code> function used for serial output of benchmark results.

4.2.4 Csolution Files

4.2.4.1 Solution File

```
solution:
  created-for: CMSIS-Toolbox@2.9.0
  description: CoreMark BenchMark Project
  cdefault:
  compiler: GCC
  select-compiler:
    - GCC
    - IAR
    - AC6

  packs:
    - pack: ARM::CMSIS
    - pack: Keil::STM32H7xx_DFP

  target-types:
    - type: H743_Device
      device: STM32H743ZITx

  build-types:
    - type: Validation
      define:
        - VALIDATION_RUN
      optimize: balanced

    - type: Performance
      define:
        - PERFORMANCE_RUN
      optimize: speed
    - type: Profile
      define:
        - PROFILE_RUN
      optimize: size

  projects:
    - project: ./coremark.cproject.yml
```

Figure 4.9: csolution file contents

The csolution file for our project defines the following information:

- **Compiler Selection:** Lists all three compilers, GCC, IAR and ARM v6.
- **Software Packs:** The device's DFP and the ARM CORE pack.
- **Target Selection:** The full name of the device to be used.

- **Build Types:** In correspondence to the CoreMark run types.

4.2.4.2 Project File

```
project:
  components:
    - component: CMSIS:CORE

  add-path:
    - modifiable
    - user_code
    - coremark
    - driver
  linker:
    - script: modifiable/gcc_linker.ld
      for-compiler: GCC
    - script: modifiable/iar_linker.icf
      for-compiler: IAR
    - script: modifiable/arm_linker.sct
      for-compiler: AC6

  groups:
    group: Startup Files
    files:
      - file : modifiable/gcc_startup.s
        for-compiler: GCC
      - file : modifiable/iar_startup.s
        for-compiler: IAR
      - file : modifiable/arm_startup.s
        for-compiler: AC6

    group: User Code Files
    files:
      - file: user_code/main.c
      - file: user_code/print_config.c

    group: CoreMark Files
    files:
      - file: coremark/core_main.c
      - file: coremark/core_portme.c
```

Figure 4.10: Extract of cproject file contents

The cproject file for our project defines the following information:

- **Software Components:** Which components to use from each software pack
- **Include Paths:** The locations of the header files to include which are all the folders in our case.
- **Linker scripts** The location of each linker script per compiler toolchain.
- **Source Files Groups:** The list of all the source files to compile.

4.2.4.3 Default File

```
default:

misc:
- for-compiler: AC6
  C:
    - -Wno-macro-redefined
    - -Wno-pragma-pack
    - -Wno-parentheses-equality
    - -Wno-license-management
    - -std=gnu11
    - -Omax
    - -flto
  ASM:
    - -masm=auto
  Link:
    - --entry=Reset_Handler
    - --info summarysizes
    - --summary_stderr
    - --diag_suppress=L6314W
    - -Omax
    - -flto

- for-compiler: GCC
  C-CPP:
    - -masm-syntax-unified
    - -fomit-frame-pointer
    - -ffunction-sections
    - -fdata-sections
    - -Ofast
    - -flto
  C:
    - -std=gnu11
  Link:
    - --specs=nano.specs
    - --specs=nosys.specs
    - -WL,-print-memory-usage
    - -WL,--gc-sections
```

Figure 4.11: Extract of cdefault file contents

The cdefault file contains compiler specific miscellaneous configurations that are propagated to all contexts and projects.

4.3 Project Generation Tool

4.3.1 Overview

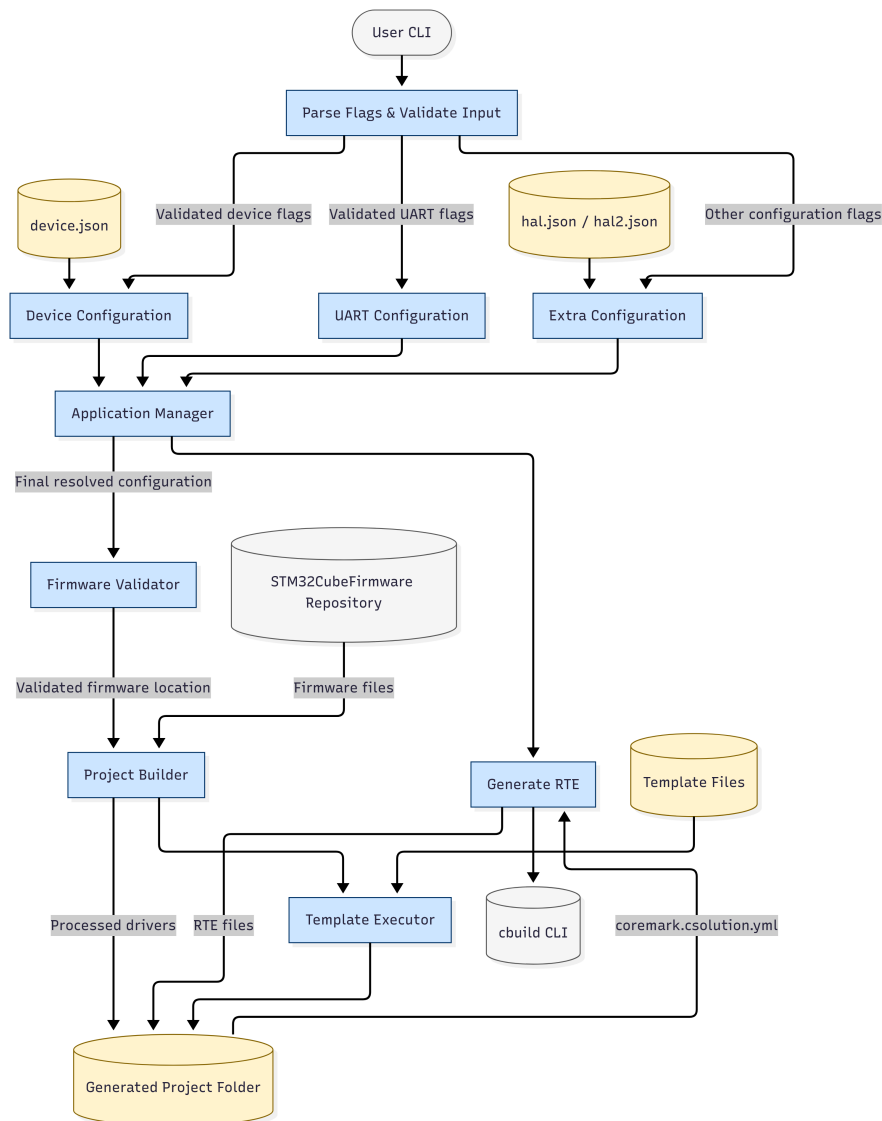


Figure 4.12: `cm_gen` DataFlow Diagram

Figure 4.12 showcases how data moves through the different components of the project generation tool, highlighting the inter-dependencies and integration of external resources and tools.

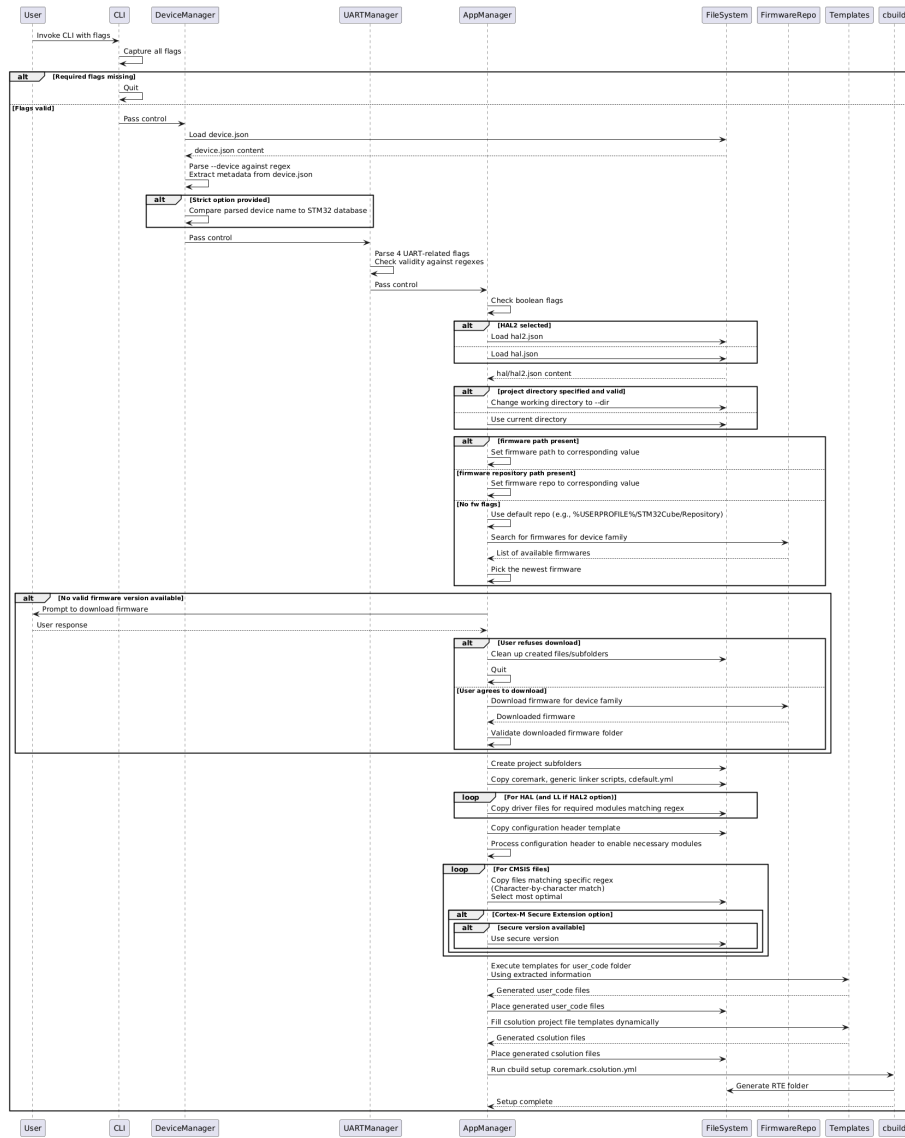


Figure 4.13: Sequence Diagram describing cm_gen's internal architecture

Figure 4.13 describes the internal workflow of the cm_gen tool, it emphasizes the modularity of the application as well as conformity to software development best practices such as the SOLID principles. These diagrams help us to understand how cm_gen works, we can divide its lifetime into two main components:

- **Data Extraction Phase:** The tool takes the user input, and extracts all the necessary information to generate a CoreMark project.
- **Build Phase:** The tool starts building the necessary components of the CoreMark project.

4.3.1.1 Resources

The project uses multiple resources to function, these are made external and not embedded within the application's source code to ensure extensibility in the future. The resources are divided into two main categories.

- **JSON Metadata Files:** These files contain information concerning devices and drivers.

- **device.json:** The device.json file consists of objects containing key value pairs that map the STM32 MCU name codification to corresponding values to be used throughout the program:
 - * **Family To Processor:** This object maps each family to its corresponding Core.
 - * **Pin Count:** The first character proceeding the device number indicates the pin count of the device, this object maps the character to the corresponding pin count.
 - * **Flash Size:** The second character proceeding the device number indicates the FLASH size of the device, this object maps the character to the corresponding FLASH size.
 - * **Packaging type:** The third character proceeding the device number indicates the packaging type of the device, this object maps the character to the corresponding packaging type.
- **hal.json/hal2.json:** These files contain all the possible modules to be enabled by the CoreMark project in order to work, they contain an array listing these modules, the separation of HAL versions is because of the difference in naming conventions and needed files.
- **Template Files:** These files are the base to be used in order to dynamically create context specific versions, they follow the original file's name with the added gotmpl extension. Below is a list of all template files:
 - main.c/h.gotmpl
 - print_config.c/h.gotmpl
 - coremark.csolution.yml.gotmpl
 - coremark.cproject.yml.gotmpl

4.3.1.2 Available Options

The first operation when invoking the tool is to parse the passed option flags in order to extract the information concerning project generation, Table 4.4 below lists all available options within the application.

Table 4.4: Project Generation Tool options

Option	Flag	Description	Example Value
Device Name	–device	The devices name the CoreMark project will be generated for	STM32H743ZITx

Continued on next page

Table 4.4 – Continued from previous page

Option	Flag	Description	Example Value
UART Handle	<code>-uart_handle</code>	The UART peripheral to use for transmitting CoreMark results	USART3
UART Transmit Pin	<code>-uart_pin</code>	The UART peripheral's transmit pin number	GPIO_PIN_3
UART Transmit GPIO	<code>-uart_gpio</code>	The UART peripheral's transmit pin GPIO	GPIO
UART Alternate Function	<code>-uart_af</code>	The UART peripheral's transmit pin alternate function	AF7
Strict Device Checking	<code>-strict</code>	If present, checks the existence of the device name against ST's device database	Boolean
HAL2	<code>-HAL2</code>	If present, use HAL2 drivers and structure instead of HAL	Boolean
Cortex-M Security Extension	<code>-cmse</code>	If present, the project is created as a secure application in the TrustZone context	Boolean
Project Directory Path	<code>-dir</code>	Specifies the the directory in which the project will be generated, defaults to the current directory	\$HOME/STM32H7_CoreMark

Continued on next page

Table 4.4 – Continued from previous page

Option	Flag	Description	Example Value
Firmware Path	<code>-fw_path</code>	Specifies the location of the device's family's STM32CubeFirmware	<code>\$HOME/stm32_h7_fw</code>
Firmware Repository Path	<code>-fw_repo</code>	Specifies the parent folder of the device's family's STM32CubeFirmware, defaults to the STM32Cube/Repository folder under the User profile's folder	<code>\$HOME/STM32Cube/Repository</code>
Processor Cache	<code>-scb_cache</code>	If present and available, enable the Cortex-M processor's cache	Boolean
HAL Cache Peripheral	<code>-hal_cache</code>	If present and available, enable ST's cache peripheral	Boolean

Each of the subsequent sections makes use of one or multiple of these options. After parsing the flags, they are passed to the data modules within the application to construct the appropriate Data classes.

4.3.2 Device Configuration

The device configuration module only requires the device name as input to construct the necessary information modeling the device, optionally if the strict option is provided, the device name validation happens in this component.

The first step is to load the device.json file. We then construct the regex allowing us to validate the device name, we join the list of keys from the device.json file's objects with the "|" symbol as a separator, the generated regex would look like

```
STM32(Family names)[A-Z0-9]{2}(Pin Counts)(FLASH Sizes)(Package Type)X[A-Z]?
```

If valid, exposes the information via a global data object.

4.3.3 UART Configuration

The UART configuration module simply checks each UART flag against a basic regex to conform to the ST drivers' naming convention and fills the data class structure with the information.

4.3.4 Extra Configuration

The Extra configuration modules checks the rest of the flags, ensuring locations exist and the compatibility of flags with the device, (e.g. you cannot enable the processor's cache with a device that doesn't support it), and the generates the last data object to be used in the execution phase.

4.3.5 Firmware Management

This part of the application checks for the validity of the firmware path within the extra configuration's amalgamated path, if it is invalid it prompts the user to download a valid version from the internet, specifically, ST distributes the firmware publicly on a GitHub repository following a standard naming convention that can be derived from the device family.

4.3.6 File Extraction

While abstractions speed up the development process and provide a reliable way to communicate with the targeted peripheral, they come at the cost of having a very high memory footprint, this side-effect might cause performance degradations as it effects memory alignments, and might even eat up entire high speed regions such as TCM and cause more cache misses. Therefore, reducing this effect to the maximum is required, and that is done in this part of the process, since we know which modules we need to add, these are the only ones whose files will be copied.

For HAL and LL files, they follow a simple naming convention that can be expressed by the following regex : `stm32(device family)_(hal|ll)_(peripheral name)(_ex)?.c/h`

For CMSIS files, which are the startup and system source files, as well as the device and system headers, the naming convention relies on the device number, and sometimes on the device metadata as well. The common symbol to use is expressed as `stm32(device family)(device number)xx`, however as mentioned, the last two characters while most commonly are "x"s can be the device's exact character, this is because sometimes the startup file or the system file might change behaviour depending on the specificities, and for devices that support TrustZone, each file is distributed in three distinct versions: secure, non-secure and a TrustZone disabled version.

4.3.7 File Processing

Even though most of the driver files can be copied directly into our CoreMark projects, some might not work out of the box, this is due to the nature of the file and what they are meant to serve within STM32 MCU ecosystem.

- **Configuration Header:** The configuration header (`stm32_hal_conf.h`) file is the equivalent to a module manager within the STM32 ecosystem, it contains the macro defines that allows us to enable the needed modules, although they can be provided in the `csolution` project files, due to their amount, they would end up clustering files with unnecessary information, therefore we opted to directly modify the template provided by ST in the STM32CubeFirmware package. This is done by reading the file into memory, reading line by line looking for matches to the modules we want to enable and uncommenting that define (by default all module defines are commented out).
- **ARM Compiler Startup File:** For HAL where startup files are not unified C language files but are instead toolchain specific assembly files, the ARM Compiler toolchain has the quirk of defining the heap and stack sizes in the startup file instead of the linker file, therefore we have to manually change the symbols to values that allows CoreMark to run without memory overflows.
- **Linker Files:** Although we use generic linker scripts for HAL projects, HAL2 based projects rely on the linker scripts provided within the STM32CubeFirmwareV2 package for the device, therefore, we manually change the stack and heap sizes defined within these files.

4.3.8 Template Generation

The last part of the execution phase is to generate files based on the template resources we have, the standard library for Golang provides a very handy package `"text/template"` that we will be using, working with this package is very straight forward, it exposes an interface to load a template file, and then executing the template consists of making use of an other interface that takes the template handle and a data object as arguments, we then save the result to the desired file.

4.3.9 Build Information Generation

After generating the project, the last step before being able to build it is to generate build information, this is in the form of the build information files of the CMSIS-Toolbox such as `cbuild.yml` and `cbuild-run.yml`, as well as the `RTE` and `.cmsis` folders containing device specific information for the build stage. This is done by invoking the CMSIS-Toolbox utility **cbuild** with the `setup` subcommand and the `csolution` file as an argument.

4.3.10 Error Handling And Resilience

With the number of variables, options, external dependencies and interactions with the file system, multiple parts of the project can fail, therefore maintaining a robust error handling system is crucial in order to handle all cases.

- **Invalid Flags:** for the crucial flags being the device name and UART information, errors will result in a non-working project, therefore the absence of these flags or their invalidity means that the program will terminate without proceeding any further, for all other flags, we simply revert to the default values and continue operating.
- **Missing Resources:** As they are an integral part of the program, not being able to load them means the application will not be able to operate.
- **Invalid Firmware:** In the case that the firmware is not available, or if for some reasons, doesn't provide the appropriate files, the program will fail gracefully, reverting all the changes it has done to the project directory.

4.3.11 Extensibility And Future Proofing

cm_gen was architected in a way that allows for extensions, as it is meant to work on any STM32 device including those that are still in development and aren't even available as chips, therefore being future-proof is a must in these circumstances. This is done in two ways throughout the application:

- **External Resources:** By providing device and library metadata in external files, we allow the user to update and customize them without having to modify the application's source code.
- **Interfaces:** Providing implementations on how to extract files and process them depends a lot on the context, by providing a generic interface that the mediator within the application invokes in an abstracted way allows us to extend this interface in the future with different implementations that adhere to new needs that aren't present at the time.
- **Dynamic templating:** The data object passed to templates is a comprehensive union of all metadata within the data objects of the application as well as extra information derived during the earlier steps of the execution phase, therefore, the template files can be modified in any way the user sees fit as long as it adheres to the dynamic variables' naming convention and making sure that these variables are provided by the data object.

Conclusion

In this chapter, we detailed the comprehensive process of implementing and automating the CoreMark benchmark for STM32 devices. The project began with porting CoreMark to STM32 microcontrollers, focusing on configuring the SysTick timer for precise performance measurements and implementing the `ee_printf` function for result reporting via UART. We then transitioned to integrating the `csolution` project structure, which streamlined project setup by defining source files, include paths, and linker scripts. The development of the `cm_gen` tool marked a significant milestone, enabling fully automated generation of CoreMark projects tailored to specific STM32 devices. This tool leveraged device metadata, UART configurations, and template-based file generation to reduce manual effort and minimize errors. Validation across all STM32 MCU families confirmed the robustness of the solution, with results benchmarked against manual setups. The modular architecture of `cm_gen`, combined with its extensible design, ensures compatibility with future STM32 devices and evolving requirements. This work establishes a scalable, efficient framework for performance benchmarking, significantly enhancing development workflows for ST's embedded systems.

Conclusion

This summer internship project at STMicroelectronics Tunis has been a comprehensive journey into embedded systems development, focusing on automating the execution of the CoreMark benchmark on STM32 microcontrollers using the Csolution project structure. Beginning with an exploration of the internship context, including ST's global presence and support solutions, the project addressed the challenges of manual benchmarking processes, critiqued existing approaches, and proposed an innovative automated solution. The theoretical foundations laid in Chapter 2 provided essential knowledge on STM32 MCUs, benchmarking principles, build processes, debugging techniques, and the Open-CMSIS-Pack ecosystem. This groundwork informed the specification of functional and non-functional requirements, structural decisions for the CoreMark project and generation tool, and the selection of hardware and software resources, as detailed in Chapter 3. The core of the project, presented in Chapter 4, involved porting CoreMark to STM32 devices, integrating it with Csolution, defining a modular project architecture, and developing the `cm_gen` CLI tool for fully automated project generation. By identifying automatable patterns and leveraging templates, metadata, and dynamic configurations, `cm_gen` ensures seamless compatibility across all STM32 families and compilers. Validation through extensive testing established a reliable baseline for performance metrics, confirming the solution's robustness. Overall, this project has successfully streamlined benchmarking workflows, reducing manual effort and enhancing efficiency for embedded developers. It demonstrates the power of automation in addressing real-world engineering challenges and positions the developed tools for future extensions, such as support for emerging STM32 devices or additional benchmarks. On a personal level, this internship has enriched my technical skills in embedded programming, tool development, and project management, while fostering a deeper appreciation for collaborative innovation in a professional setting. I am grateful for the opportunity and look forward to applying these insights in future endeavors.

Abstract

This report describes the work carried out during my summer internship at STMicroelectronics, focused on platform-agnostic benchmarking. The main objective was to run Coremark on STM32 devices using Csolution project structure. To achieve this, I put to use my knowledge in software and embedded development, C code compilation and code generation, using tools such as CMSIS-Toolbox, C and Golang. The results helped to drastically reduce the time required to setup and run Coremark projects using multiple compilers, as well as generating the most optimal results. This internship also allowed me to strengthen my skills in the compilation process, embedded projects architecture design and gain insight into the specific challenges of embedded systems engineering.

Keywords : C/C++, Golang, Embedded Systems, STM32, Compilation, Code Generation.

Résumé

Ce rapport décrit les travaux réalisés lors de mon stage d'été chez STMicroelectronics, axé sur l'analyse comparative indépendante de la plateforme. L'objectif principal était d'exécuter Coremark sur des microcontrôleurs STM32 en utilisant la structure de projet Csolution. Pour ce faire, j'ai mis à profit mes connaissances en développement logiciel et embarqué, en compilation et génération de code C, à l'aide d'outils tels que CMSIS-Toolbox, C et Golang. Les résultats ont permis de réduire considérablement le temps nécessaire à la configuration et à l'exécution des projets Coremark avec plusieurs compilateurs, tout en optimisant les résultats. Ce stage m'a également permis de renforcer mes compétences en compilation, en conception d'architecture de projets embarqués et de mieux comprendre les défis spécifiques de l'ingénierie des systèmes embarqués.

Mots clés : C/C++, Golang, Systèmes embarqués, STM32, Compilation, Génération de Code.